CubeGraph: A Multi-Resolution Graph Index for Approximate k-Nearest Neighbor Search with Geometric Filters

Mingyu Yang
HKUST (GZ) & HKUST
China
myang250@connect.hkust-gz.edu.cn

Wentao Li
University of Leicester
United Kingdom
wl226@leicester.ac.uk

Wenxuan Xia
HKUST (GZ)
China
wxia248@connect.hkust-gz.edu.cn

Wei Wang
HKUST (GZ) & HKUST
China
weiwcs@ust.hk

ABSTRACT

Approximate k-nearest neighbor (AkNN) search retrieves the k vectors closest to a query vector. In many applications, vectors have numerical attributes, such as location and time. This motivates filtered AkNN search, which retrieves nearest neighbors satisfying attribute constraints. Existing methods typically support regular geometric filters (e.g., hyperrectangles) and follow a two-stage process: generating candidate vectors that satisfy the filter and refining the candidates to identify nearest neighbors. However, real-world filters can define irregular or arbitrary regions, such as geographic boundaries for points of interest (POIs). Such filters are poorly supported by existing methods due to an inherent trade-off: Extensive candidate generation reduces refinement overhead but increases computational cost, whereas lightweight candidate generation incurs expensive refinement.

To address these challenges, we propose CubeGraph, a multi-resolution graph index for AkNN search with geometric filters. CubeGraph partitions the attribute space into a fixed-depth hierarchy of hyperrectangular cells. It constructs local graph indexes within cells and introduces cross-cell connections to improve connectivity. At query time, CubeGraph selects a hierarchy level based on the filter's geometric characteristics to balance candidate generation and refinement costs. To further accelerate refinement, CubeGraph employs two search modes: predetermined search for simple filters and on-the-fly search for complex filters. Experiments on four datasets with up to 10 million vectors demonstrate that CubeGraph supports diverse geometric filters and achieves superior recall–throughput trade-offs over existing methods.

PVLDB Reference Format:
Mingyu Yang, Wentao Li, Wenxuan Xia, and Wei Wang. CubeGraph: A Multi-Resolution Graph Index for Approximate k-Nearest Neighbor Search with Geometric Filters. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/mingyu-hkustgz/CubeGraph.

1 INTRODUCTION

In many real-world AI applications, objects such as images and text are mapped into high-dimensional vector embeddings that capture their semantic information [27, 29]. Given a dataset $\mathcal{D}$ of vector embeddings and a query vector q, k-nearest neighbor (kNN) search retrieves the k vectors in $\mathcal{D}$ closest to q. Yet, exact kNN search is computationally expensive in high-dimensional spaces due to the curse of dimensionality [14]. Thus, **approximate kNN (AkNN) search** is widely adopted, which retrieves approximate kNN results to improve query efficiency at the cost of some loss in accuracy [4].

AkNN search is widely used in applications such as retrieval-augmented generation for large language models and recommendation systems. In many applications, objects are also associated with numerical attributes, such as location and timestamps [2]. Users may therefore seek objects that are both similar to a query vector and satisfy specified attribute constraints, such as a timestamp falling within a given range. To support such queries, **filtered AkNN search** [21] incorporates an attribute filter ϕ and retrieves the approximate k nearest neighbors among objects satisfying ϕ. This enables more flexible retrieval, such as finding images captured within a specified time range.

New Problem. Despite the appeal of filtered AkNN search, existing studies typically restrict attribute filters ϕ to regular geometric regions, such as one-dimensional intervals or low-dimensional hyperrectangles. This is largely because efficiently supporting even these relatively simple filters remains challenging [2, 21, 43, 53]. However, emerging real-world applications increasingly require more complex filters involving multiple, potentially correlated attributes. For example, users may seek objects semantically similar to a query image whose attributes fall within a geographic polygon in two-dimensional (2D) space, a spatio-temporal region in 3D space, a higher-dimensional hyperrectangle, or a composite region with both inclusion and exclusion constraints. Such requirements extend beyond the restricted filter types supported by existing methods.

We therefore extend existing studies by investigating AkNN search with **geometric filters**. Unlike conventional attribute filters restricted to regular geometric regions, geometric filters allow users to specify arbitrary regions in numerical attribute spaces, including composite regions defined by inclusion and exclusion constraints. Formally, given a dataset $\mathcal{D}$ of vector embeddings with numerical attributes, a query vector $\mathbf{q}$, and a geometric filter ϕ, AkNN search with geometric filters retrieves the approximate k nearest neighbors of $\mathbf{q}$ among objects whose attributes satisfy ϕ. The filter ϕ may define an interval, hyperrectangle, circle, polygon, or arbitrary inclusion–exclusion region in the attribute space. We illustrate the expressiveness of this problem through two applications.

Urban Event Retrieval. A city management platform stores geo-tagged texts, images, and videos, each represented by a vector embedding with a 1D timestamp and 2D location attributes. Given

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

a query vector $\mathbf{q}$ representing an event such as *flooded streets*, an analyst seeks semantically similar objects captured within a specified time window and an irregular flood-affected region. Such a filter defines an irregular region in the 3D attribute space, which cannot be precisely represented by conventional axis-aligned range filters. Approximating it with a bounding box may include objects outside the specified region, requiring additional filtering.

Regional Multimedia Retrieval. A multimedia database $\mathcal{D}$ stores geo-tagged images and videos, each represented by a vector embedding with a 1D timestamp and 2D location attributes. Given a query image $\mathbf{q}$, a user seeks visually similar objects captured within a specified time interval and a complex geographic polygon, while excluding restricted subregions such as airports or military zones. Such composite *inclusion–exclusion* filters enable users to express complex retrieval requirements that cannot be directly captured by conventional range filters.

Existing Solutions. Despite its greater expressive power, AkNN search with geometric filters is challenging because it requires jointly handling high-dimensional vector similarity search and complex geometric filters over multidimensional attributes. A natural approach is to adapt existing filtered AkNN methods designed for regular attribute filters, which can be broadly classified into two categories: *filtering-based* and *partition-based* methods.

Filtering-based Methods. These methods build on conventional AkNN indexes that do not incorporate attribute information. In particular, graph-based indexes (e.g., HNSW [27]) are widely adopted due to their state-of-the-art query performance [4, 9, 38]. Such indexes represent vectors as graph nodes connected by edges, enabling efficient AkNN search through graph traversal. To support attribute filters, existing methods typically adopt either PreFiltering or PostFiltering. PreFiltering excludes nodes that violate the filter during graph traversal, but this may disrupt graph connectivity and degrade search accuracy. In contrast, PostFiltering performs graph traversal without considering attribute constraints and filters out invalid candidates afterward, incurring redundant distance computations and reducing query efficiency when many candidates fail the filter. Methods such as ACORN [29] improve connectivity under filtering by constructing denser graphs with additional edges, but incur higher index storage costs, particularly when supporting complex geometric filters.

Partition-based Methods. These methods incorporate attribute information by partitioning the attribute space to organize dataset vectors and enable pruning during filtered AkNN search [7, 8, 13, 16, 28, 29, 34, 40–43, 45, 47, 50, 52, 53]. For one-dimensional (1D) attributes (e.g., timestamps), methods such as WindowFilter [8], iRange [43], ESG [45], and WoW [40] organize objects using segment-tree-like structures over the attribute space. However, objects may be stored at multiple tree levels, leading to $O(N \log N)$ storage in the worst case, where N is the number of objects in $\mathcal{D}$. For multidimensional attributes, Mesh [33] and KHI [49] employ R-tree-based and KD-tree-based spatial indexes, respectively, to partition the attribute space and organize objects within index nodes for query processing.

Challenges. In summary, existing methods for filtered AkNN search, whether attribute-agnostic or attribute-aware, generally follow a two-stage framework: candidate generation and result refinement. They first generate a candidate set containing all filter-matching objects and then identify the nearest neighbors from this set. For example, filter-based methods such as PreFiltering eagerly eliminate non-matching objects before nearest neighbor search, whereas PostFiltering defers filter verification to the refinement stage. Partition-based methods employ structures such as segment trees or spatial trees to prune regions containing no matching objects, followed by result refinement.

For AkNN search with geometric filters, existing methods face an inherent trade-off between candidate generation and result refinement costs. (1) Lightweight candidate generation, as in PostFiltering, which ignores filters during search, produces many non-matching candidates, resulting in expensive refinement. (2) Conversely, aggressive filtering, as in PreFiltering, eliminates non-matching objects early to reduce refinement costs. However, evaluating complex geometric filters is computationally expensive, and filter-matching objects may be scattered across disconnected regions of the graph index, making them difficult to retrieve. Partition-based methods face a similar trade-off. Selecting coarse-grained nodes near the root of a spatial tree reduces candidate generation costs but includes many non-matching objects. Traversing deeper into the tree reduces such objects at the expense of higher candidate generation costs. Moreover, fine-grained partitions fragment the search index, potentially degrading search accuracy.

Our Solution. To address these limitations, we propose CubeGraph, a multi-resolution graph index for AkNN search with geometric filters. CubeGraph comprises a hierarchy of L layers, each partitioning the attribute space into hyperrectangular cells at different resolutions. Upper layers contain fewer, coarser cells, while lower layers progressively subdivide them into more, finer cells. Each cell contains objects whose attributes fall within its boundaries, and a graph index is constructed at each layer.

(1) To balance candidate generation and refinement costs, CubeGraph adaptively selects a layer based on the geometric characteristics of the query filter. Coarser layers require fewer cell intersection tests but introduce more non-matching candidates, whereas finer layers reduce refinement costs at the expense of more intersection tests. This adaptive selection balances the two costs according to the query filter. To preserve search accuracy, CubeGraph further augments each layer with cross-cell edges connecting objects in adjacent cells. These edges enable traversal across cells without reconstructing the graph for each query filter, even when matching objects are distributed across different regions of the index.

(2) Once a layer is selected, CubeGraph offers two search strategies to further balance candidate generation and refinement costs at the object level. For simple geometric filters, where cell–filter intersections can be efficiently determined, *predetermined search* identifies all intersecting cells before graph traversal. This improves refinement accuracy by covering cells that may be unreachable through graph traversal, but incurs additional candidate generation costs from examining all cells. For complex geometric filters, where identifying intersecting cells is expensive, *on-the-fly search* dynamically evaluates and activates cells during graph traversal. This reduces candidate generation costs but may compromise refinement accuracy by missing unreachable cells. To exploit their

complementary strengths, CubeGraph combines both strategies to achieve a better balance between efficiency and accuracy.

Contributions. Overall, CubeGraph integrates adaptive layer selection, prebuilt face-adjacent cell links for improved result refinement, and complementary search strategies. Although hierarchical spatial partitioning is a well-established technique, our contribution lies in adapting this structure to directly address the limitations of existing methods for AkNN search with geometric filters. Our main contributions are summarized as follows.

Limitation Analysis (§ 2–3). We extend filtered AkNN search beyond regular attribute-space constraints to support arbitrary geometric filters. We analyze existing methods and identify their inherent limitations in balancing candidate generation and result refinement.

Our New Index (§ 4). We propose CubeGraph, a multi-resolution graph index that organizes the attribute space into a hierarchy of progressively finer partitions, with each cell maintaining a local graph and cross-cell connections. At query time, CubeGraph adaptively selects a hierarchy layer based on the query filter to balance candidate generation and refinement costs. We further develop predetermined and on-the-fly search algorithms tailored to simple and complex geometric filters, respectively.

Index Construction and Maintenance (§ 5). We develop efficient algorithms for constructing local graphs and cross-cell edges at each layer, while reusing graph structures across layers to reduce construction overhead. We also propose efficient index maintenance algorithms to support object insertions and deletions.

Experiments (§ 6). We conduct extensive experiments on four real-world datasets containing up to 10 million vectors. The results show that CubeGraph supports diverse geometric filters and achieves superior search performance compared with existing methods.

Due to space limitations, additional proofs and experimental results are provided in the Appendix.

2 PRELIMINARIES

We first formally define the approximate k-nearest neighbor (AkNN) search problem with geometric filters in § 2.1, followed by the necessary technical background in § 2.2.

2.1 Problem Statement

Let $\mathcal{D} = \{o_i\}_{i=1}^{N}$ be a dataset of N objects, where each object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$ consists of a d-dimensional embedding $\mathbf{x}_i \in \mathbb{R}^d$ capturing semantic information and an m-dimensional attribute vector $\mathbf{s}_i \in \mathbb{R}^m$ specifying query constraints. For example, an image may be represented by a CLIP embedding and a spatiotemporal attribute vector. We focus on numerical attributes with meaningful spatial coordinates, leaving categorical attributes for future work.

Definition 2.1. A *filter* is a Boolean function $\phi : \mathcal{S} \to \{0, 1\}$ over an m-dimensional attribute space $\mathcal{S} \subseteq \mathbb{R}^m$. An object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$ satisfies ϕ if and only if $\phi(\mathbf{s}_i) = 1$.

Without loss of generality, we assume that filters are defined over all m attribute dimensions. A filter involving only a subset of dimensions can be equivalently expressed over $\mathcal{S}$ by leaving the remaining dimensions unconstrained. We define the set of objects satisfying a filter as follows.

Definition 2.2. Given a dataset $\mathcal{D}$ and a filter ϕ, the *filtered set* is $\mathcal{D}_\phi = \{\, o_i = (\mathbf{x}_i, \mathbf{s}_i) \in \mathcal{D} \mid \phi(\mathbf{s}_i) = 1 \,\}$.

Geometrically, a filter ϕ defines a region in the attribute space, which may be a hyperrectangle, polygon, temporal window, or Boolean combination of such constraints. Let $B(\phi)$ denote the minimum axis-aligned bounding box enclosing this region. We define the *characteristic length* $len(\phi)$ as the maximum side length of $B(\phi)$. Thus, $\mathcal{D}_\phi$ contains exactly those objects whose attribute vectors lie within the region defined by ϕ.

Example 2.3. Consider a dataset $\mathcal{D}$ of four objects, each represented by an embedding $\mathbf{x}_i$ and an attribute vector $\mathbf{s}_i$. Suppose a filter ϕ defines an irregular region whose bounding box $B(\phi)$ has side lengths b_1 and b_2, where $b_1 > b_2$. Its characteristic length is therefore $len(\phi) = b_1$. If only o_1 and o_3 lie within the region, the filtered set is $\mathcal{D}_\phi = \{o_1, o_3\}$.

Problem Definition. Given a dataset $\mathcal{D}$ and a query vector $\mathbf{q}$, approximate k-nearest neighbor (AkNN) search aims to retrieve the k objects in $\mathcal{D}$ closest to $\mathbf{q}$. When a filter ϕ is specified, filtered AkNN search restricts the search to the filtered set $\mathcal{D}_\phi$. Every returned object must satisfy ϕ, while nearest-neighbor retrieval is approximate and evaluated by recall. The target result size is $\min(k, |\mathcal{D}_\phi|)$, although a limited search budget may yield fewer results. We formally define the problem as follows.

Definition 2.4. Given a dataset $\mathcal{D}$ and a query $Q = (\mathbf{q}, \phi)$, *AkNN search with geometric filters* returns a result set $\mathcal{R} \subseteq \mathcal{D}_\phi$ that approximates the k nearest neighbors of $\mathbf{q}$ within $\mathcal{D}_\phi$.

Unlike existing studies that primarily consider axis-aligned hyperrectangles or hyperspheres, we impose no restrictions on the shape of the geometric filter ϕ. Although general Boolean predicates can express the same eligible set, we explicitly exploit spatial relationships in the numerical attribute space to facilitate efficient search. Hereafter, we use *region* and *predicate* interchangeably when referring to geometric filters. Unless otherwise specified, we measure vector similarity using Euclidean distance, although other measures, such as inner product, can be supported by the underlying AkNN index. For simplicity, we omit k from the query notation. Table 1 summarizes the notation used throughout the paper.

Example 2.5. Consider a query vector $\mathbf{q}$ with $k = 1$. Suppose o_2 is its nearest neighbor in $\mathcal{D}$ but does not satisfy ϕ, where $\mathcal{D}_\phi = \{o_1, o_3\}$. If o_3 is closer to $\mathbf{q}$ than o_1, then o_3 is the nearest eligible neighbor and thus the desired result.

2.2 Technical Foundations

We introduce two technical foundations relevant to AkNN search with geometric filters: plain AkNN search without filters and index merging over partitioned vector datasets.

Plain AkNN Search. Plain AkNN search retrieves approximate nearest neighbors in a high-dimensional vector space without attribute constraints. Existing methods can be broadly categorized into hashing-, quantization-, and graph-based approaches. Hashing methods, such as locality-sensitive hashing (LSH) [12], map nearby vectors to the same buckets with high probability and answer queries by probing candidate buckets. Quantization methods [3, 5, 10, 11, 19, 46] compress vectors into compact representations, reducing storage costs and enabling efficient approximate

Table 1: Summary of Notations

Notation	Description
$\mathcal{D}$	Dataset containing N objects
o_i	Object $o_i = (\mathbf{x}_i, \mathbf{s}_i)$
N	Number of objects
d	Embedding dimensionality
m	Attribute dimensionality
$\mathbf{x}_i$	Embedding vector of object o_i
$\mathbf{s}_i$	Attribute vector of object o_i
$\mathcal{S}$	Attribute space
$\mathbf{q}$	Query vector
ϕ	Geometric filter over $\mathcal{S}$
$B(\phi)$	Tight axis-aligned bounding box enclosing $\mathcal{D}_\phi$
$len(\phi)$	Maximum side length of $B(\phi)$
k	Number of nearest neighbors
$\mathcal{D}_\phi$	Objects satisfying ϕ
$\mathcal{R}$	Approximate top-k result set

distance computation. Graph-based methods construct proximity graphs over the dataset and perform graph traversal to identify nearest neighbors. Representative indexes include HNSW [27], NSG [9], HVS [24], tMRNG [30], DiskANN [15], and others [20, 36, 37, 48]. We adopt graph-based indexes as the underlying structure of our method due to their strong search performance.

Index Merging. Graph connectivity is crucial for accurate and efficient graph-based search. Index merging combines multiple independently constructed graph indexes into a unified searchable structure. It is particularly useful in disk-based, distributed, and heterogeneous vector search systems, where constructing or maintaining a monolithic index can be expensive or impractical [15, 52]. A common approach partitions the dataset, constructs an index for each partition, and introduces overlapping objects or cross-partition edges to preserve connectivity. For example, DiskANN employs overlapping partitions to support billion-scale vector search under memory constraints [15]. Efficient HNSW Merging combines multiple HNSW indexes into a unified graph [17]. FGIM accelerates graph merging using an improved NN-Descent algorithm [1], while RNSM establishes cross-partition connections by identifying nearest neighbors across partitions [18]. These methods highlight the importance of cross-partition connectivity in preserving graph navigability after merging.

3 PROBLEM ANALYSIS

AkNN search with geometric filters can potentially leverage existing filtered AkNN methods, which primarily support axis-aligned ranges or hyperrectangles. We first review these methods in § 3.1 and then discuss the challenges of extending them to arbitrary geometric filters in § 3.2, motivating our proposed approach.

3.1 Existing Solutions

We categorize existing methods for filtered AkNN search into *filtering-based* and *partition-based* approaches. The former do not incorporate attributes into index construction, whereas the latter explicitly consider attributes when building indexes.

Filtering-based Methods. Filtering-based methods maintain a general-purpose index for plain AkNN search and apply attribute constraints during or after graph traversal. Two representative strategies are PreFiltering and PostFiltering [13]. PreFiltering prunes objects that violate the attribute filter during traversal, avoiding unnecessary exploration of filter-mismatched objects. However, this may disrupt graph connectivity under selective filters, degrading search accuracy. In contrast, PostFiltering traverses the original graph without restricting exploration to filter-matched objects, preserving graph connectivity and search accuracy. Yet, it incurs overhead by visiting numerous objects that do not satisfy the filter.

Several methods improve these strategies for specific types of attribute filters. For categorical filters (e.g., brand names), Filtered-DiskANN [13] constructs category-aware subgraphs and answers queries using the corresponding subgraphs. For low-dimensional numerical attributes, NHQ [35] incorporates both attribute proximity and vector similarity into graph construction. More recently, ACORN [29] augments graph index HNSW with additional edges and dynamic neighbor expansion to facilitate traversal under attribute constraints. Notably, ACORN supports arbitrary filters, including geometric filters, through a membership mask. However, identifying filter-matched neighbors requires examining numerous candidates, resulting in substantial traversal overhead. Moreover, graph connectivity may still deteriorate when filter-matched objects are sparsely distributed. Our experiments show that ACORN fails to achieve the target recall under such conditions.

Partition-based Methods. Partition-based methods divide the attribute space into subspaces and construct a local graph index (e.g., HNSW) for each subspace, typically organizing these indexes into hierarchical structures. For one-dimensional range filters, methods such as SeRF [53], WindowFilter [8], iRangeGraph [43], ESG [45], and WoW [40] organize subspaces and their graph indexes using segment-tree-like structures. A query range can be decomposed into $O(\log N)$ segments, which are processed using different search strategies. For example, iRangeGraph constructs a query-specific graph by merging elemental graphs stored in the tree. However, replicating objects across tree levels can incur $O(N \log N)$ space overhead, and these methods are primarily designed for one-dimensional range filters.

For multi-dimensional attributes, spatial hierarchies such as R-trees [6] and KD-trees [32] partition the attribute space into subregions. For example, Mesh [33] constructs an R-tree in which each node maintains a local graph index over the objects in its subregion. During query processing, it identifies tree nodes overlapping the filter region and performs independent searches on the corresponding local indexes. KHI [49] instead partitions the attribute space using a KD-tree, with graph indexes maintained at tree nodes. Given a query, KHI first identifies filter-matched objects as entry points and then dynamically constructs a search graph by traversing the tree from the root to relevant nodes, connecting filter-matched objects as neighbors to facilitate graph traversal.

3.2 Challenges

When extended to geometric filters, existing methods face an inherent trade-off between candidate generation cost and graph connectivity, which directly affects search accuracy.

(1) Limitations of Filtering-based Methods. Filtering-based methods generally impose no restrictions on filter shapes and can thus support arbitrary geometric filters. However, they are not specifically designed for such filters and suffer from an inherent trade-off. PreFiltering excludes filter-mismatched objects during traversal, reducing the number of explored candidates at the cost of potentially expensive filter evaluations. More importantly, pruning these objects can severely disrupt graph connectivity, particularly

when filter-matched objects are sparse, leading to low search accuracy. In contrast, PostFiltering preserves graph connectivity by traversing filter-mismatched objects without evaluating their filter membership during traversal. However, this results in numerous unnecessary explorations, increasing search costs. Methods such as ACORN mitigate this trade-off through graph augmentation and dynamic neighbor expansion. Nevertheless, they incur substantial filter evaluation overhead when identifying eligible neighbors and may still suffer from poor connectivity when filter-matched objects are sparsely distributed.

(2) Limitations of Partition-based Methods. Partition-based methods explicitly incorporate attribute information into index construction, making them more suitable for filtered AkNN search. However, they still face limitations when handling complex geometric filters. For example, Mesh identifies tree nodes overlapping the filter region and searches their local graph indexes independently. This fragmentation limits graph connectivity across subregions, potentially degrading search accuracy. In contrast, KHI improves connectivity by dynamically constructing a query-specific graph. However, this requires extensive filter membership checks and on-the-fly graph construction, resulting in substantial computational overhead.

Key Challenges. In summary, existing methods face two fundamental challenges when handling geometric filters. (1) *Maintaining graph connectivity.* Methods such as PreFiltering and Mesh restrict traversal to filter-matched objects or local subgraphs, potentially disrupting connectivity and degrading search accuracy. (2) *Reducing search overhead.* Methods such as PostFiltering preserve connectivity by exploring numerous filter-mismatched objects, whereas ACORN and KHI improve connectivity through dynamic graph construction or expansion at the cost of extensive filter membership checks. The key challenge is therefore to maintain a well-connected search graph while minimizing query processing costs. Specifically, an effective solution should avoid both the unnecessary exploration of filter-mismatched objects in PostFiltering and the expensive on-the-fly graph construction or expansion in ACORN and KHI. We present our solution in § 4.

4 OUR PROPOSED INDEX AND QUERY PROCESSING

This section presents the proposed CubeGraph index. We first describe its structure in § 4.1, followed by the query processing algorithm in § 4.2 and its complexity analysis in § 4.3.

4.1 The CubeGraph Index

To overcome the limitations of existing methods, which suffer from either limited graph connectivity or high graph construction costs (see § 3.2), we propose a new index, CubeGraph. The key idea is to enable adaptive graph selection that (1) reduces the number of filter-mismatched objects while (2) preserving graph connectivity.

Index Structure. Let $\mathcal{D} = (\mathbf{x}_i, \mathbf{s}i)i = 1^N$, where $\mathbf{x}_i$ denotes a vector embedding and $\mathbf{s}_i \in \mathbb{R}^m$ represents its numerical attributes. After normalizing each attribute dimension to $[0, 1]$, CubeGraph defines the attribute space as $\mathcal{B} = [0, 1]^m$. To support multi-resolution indexing, CubeGraph organizes the attribute space into a hierarchy of layers, where upper layers partition the space into coarse regions and lower layers progressively refine these regions into smaller ones. This hierarchical design allows each query to select an appropriate layer, balancing filter selectivity and graph connectivity.

Specifically, CubeGraph consists of L layers, where L is a user-defined parameter. At layer ℓ ($0 \le \ell < L$), the attribute space $\mathcal{B}$ is uniformly partitioned into $(g_\ell)^m$ cubes of subspaces (i.e., axis-aligned boxes), where $g_\ell = 2^{\ell+1}$ and each cube has side length $w_\ell = 1/g_\ell$. Thus, higher layers contain fewer, larger cubes, whereas lower layers contain more, smaller cubes. A cube is identified by its grid coordinate $\mathbf{a} = (a_1, \ldots, a_m) \in \{0, \ldots, g_\ell - 1\}^m$ and covers $\prod_{r=1}^{m} [a_r w_\ell, (a_r + 1)w_\ell)$, with the upper boundary 1 included in the last interval of each dimension. Each object belongs to exactly one cube at every layer.

Two cubes $c_{\ell,\mathbf{a}}$ and $c_{\ell,\mathbf{b}}$ are **face-adjacent** if $|\mathbf{a} - \mathbf{b}|1 = 1$, meaning they differ by one grid step in exactly one attribute dimension and share an $(m-1)$-dimensional face. For each non-empty cube, CubeGraph constructs a bounded-degree local AkNN graph (i.e., HNSW) over its objects, with edges connecting each object to its local neighbors. To preserve connectivity across cube boundaries, each object additionally maintains edges to its Mcross nearest neighbors in each occupied face-adjacent cube. These cross-cube edges connect adjacent occupied cubes, while global connectivity depends on the connectivity of both the local graphs and the adjacency graph of occupied cubes. We formally define the L-layer CubeGraph index as follows.

*Definition 4.1 (*CubeGraph *Index).* Given a dataset $\mathcal{D}$, the CubeGraph index is defined as

$$\text{CubeGraph} = \{(C_\ell, \mathcal{G}_\ell^{\text{intra}}, \mathcal{G}_\ell^{\text{cross}})\}_{\ell=0}^{L-1},$$

where C_ℓ is the directory of non-empty cubes at layer ℓ, $\mathcal{G}_\ell^{\text{intra}}$ stores the local AKNN graph of each cube in C_ℓ, and $\mathcal{G}_\ell^{\text{cross}}$ stores the cross-cube edges between face-adjacent cubes.

Example 4.2. The upper-left panel of Fig. 1 shows successive 2×2, 4×4, and finer partitions of a 2D attribute space. Each object belongs to one cube per layer; local graphs and cross-cube edges connect objects within and across cubes. The lower-left panel extends the hierarchy to 3D, where an interior cube has six face neighbors and a corner cube has three.

Memory Layout. We next describe the physical memory layout of the logical components in Definition 4.1. At each layer ℓ, the cube directory C_ℓ is implemented as a sparse array that materializes only non-empty cubes and maps each cube coordinate to a contiguous range of object records. Since each object belongs to exactly one cube per layer, all records required for intra-cube traversal are stored contiguously. As illustrated in Fig. 2, each object record stores the intra-cube neighbor list, cross-cube neighbor lists, the object ID, and its attribute metadata. This layout follows the graph search access pattern (described next). Traversal first scans intra-cube neighbors in $\mathcal{G}_\ell^{\text{intra}}$; upon reaching a cube boundary, it accesses only the cross-cube neighbor group corresponding to the target face-adjacent cube in $\mathcal{G}_\ell^{\text{cross}}$. Accordingly, cross-cube neighbors are grouped by adjacent cube rather than merged into a single list. The same record layout is used at every layer, enabling a uniform access pattern across all layers while differing only in cube granularity.

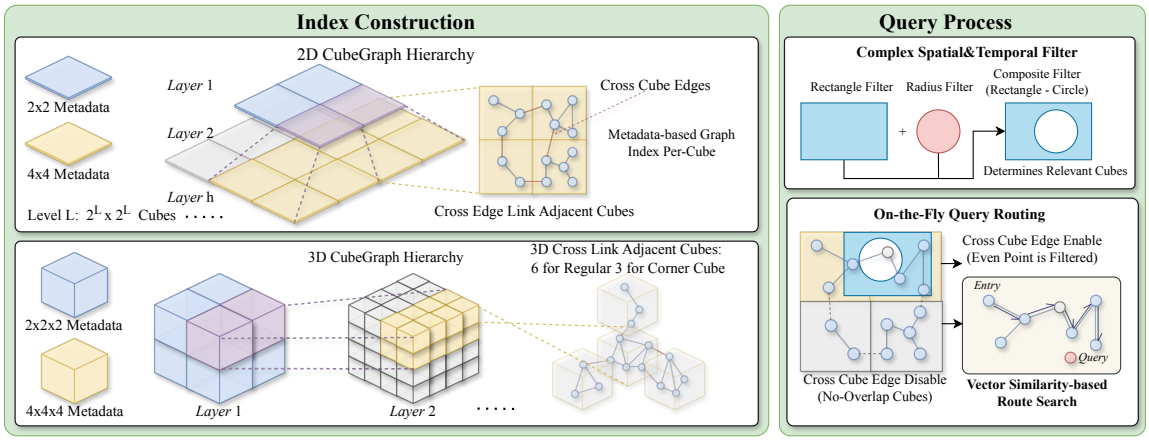


Figure 1: CubeGraph overview. Left: 2D/3D cube hierarchies and local graphs linked by cross-cube edges (only a subset is drawn). Right: cube selection and traversal for a rectangle-minus-circle predicate. Figure layers are numbered from one; algorithms use zero-based indices.

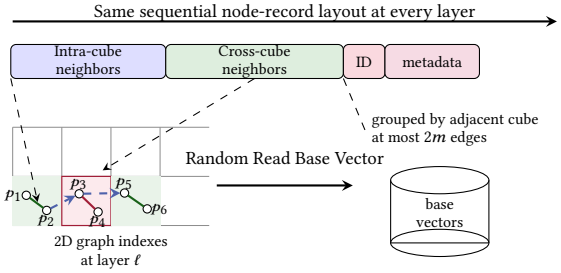


Figure 2: Physical memory layout of the CubeGraph **index.**

Example 4.3. Consider object p_2 in Fig. 2. At each layer, its intra-cube neighbor list stores the local neighbor p_1, since both objects belong to the same cube. Its cross-cube neighbor list stores neighbors in the right face-adjacent cube (e.g., p_5). The record also stores the object ID and attribute vector.

Index Size. We next analyze the space complexity of CubeGraph under the memory layout described above.

Theorem 4.4. *Assume that* CubeGraph *materializes only non-empty cubes, and the maximum intra-cube degree M and maximum cross-cube degree M_{cross} are bounded by constants. Then the space complexity of* CubeGraph *is $O(LNm)$, where L is the hierarchy depth, N is the number of objects, and m is the attribute dimensionality.*

The bound counts index records and adjacency, with fixed degree parameters; embedding vectors are stored separately. A bounded L controls replication but is not unique to cubes: other hierarchies can also be truncated. Our default caps L at six and stops refinement when the occupancy rule would yield fewer than 50 objects per cube. This trades resolution against storage; Appendix A.9 measures the effect of changing L.

4.2 Query Processing

CubeGraph organizes the objects in dataset $\mathcal{D}$ into a hierarchy of local graphs connected by cross-cube edges (see Definition 4.1). Each query uses one resolution of this hierarchy. Given a query $(\mathbf{q}, \phi)$, where $\mathbf{q}$ is the query vector and ϕ is a region predicate over

Algorithm 1: CubeGraph Query Processing

Input: CubeGraph index, query vector $\mathbf{q}$, region predicate ϕ, result size k, search budget ef
Output: Top-k objects satisfying ϕ

1 $r \leftarrow \text{len}(\phi)$;
2 Select layer ℓ^* such that $r/2 < w_{\ell^*} \le r$; if none exists, choose the closest boundary layer;
3 **if** *ϕ supports efficient cube–predicate intersection tests* **then**
4 **return** PredeterminedSearch$(\mathbf{q}, \phi, k, \ell^*, ef)$;
5 **else**
6 $c_0 \leftarrow$ the cube at layer ℓ^* containing a representative point of ϕ;
7 **return** OnTheFlySearch$(\mathbf{q}, \phi, k, \ell^*, c_0, ef)$;

the attributes, query processing first selects the appropriate hierarchy level among the L layers of CubeGraph, and then performs graph search on the selected layer. Accordingly, query processing consists of two phases: (1) layer selection and (2) graph search. Algorithm 1 summarizes the overall procedure. We describe these two phases below.

Phase 1: Layer Selection. Given a query $(\mathbf{q}, \phi)$, CubeGraph first selects the appropriate layer among the L hierarchy levels. The selection is based on the characteristic length $r = \text{len}(\phi)$ of the query region, defined as the maximum side length of the bounding box enclosing ϕ. At layer ℓ, the cube width is $w_\ell = 1/g_\ell$, where $g_\ell = 2^{\ell+1}$. Thus, layer selection compares the query scale with the cube width. Specifically, as shown in Algorithm 1, we first compute the characteristic length $r = \text{len}(\phi)$ in the normalized attribute space (Line 1). CubeGraph then selects the layer ℓ^* whose cube width best matches the query scale (Line 2):

$$\frac{r}{2} < w_{\ell^*} \le r.$$

This scale heuristic limits the number of intersecting cubes when a matching layer exists. It does not use occupancy or guarantee that a cube fits inside the region; narrow, hollow, or disconnected predicates can still admit many invalid candidates. If no layer satisfies the above condition, the closest boundary layer is selected (i.e., the layer whose cube width is just larger than r or just smaller than $r/2$). Moreover, since w_ℓ decreases monotonically with ℓ, ℓ^* can be

Algorithm 2: Predetermined Search

Input: Query $\mathbf{q}$, predicate ϕ, result size k, layer ℓ, budget ef
Output: Top-k results satisfying ϕ

1 $R \leftarrow \emptyset$;
2 Identify the set of cubes $C = \{c_1, \ldots, c_t\}$ intersecting ϕ;
3 Construct an adjacency bitmap B with $B[c] = 1$ for every $c \in C$;
4 Initialize a priority queue Q with the entry points of all cubes in C;
5 **while** $Q \neq \emptyset$ ***and*** *($|R| < k$* ***or*** *the closest candidate in Q is closer than the k-th result in R)* **do**
6 $\quad p \leftarrow$ pop the closest candidate from Q;
7 $\quad$ **for each** *neighbor u of p via an intra- or cross-cube edge* **do**
8 $\quad\quad$ **if** (p, u) is a cross-cube edge **and** $B[u.\text{cube}] = 0$ **then continue**;
9 $\quad\quad$ **if** $\phi(\mathbf{s}_u) = 1$ **then** insert u into R;
10 $\quad\quad$ **if** u has not been visited **then** insert u into Q;
11 $\quad\quad$ Retain only the best ef candidates in Q;
12 **return** the top-k results in R;

Algorithm 3: On-the-Fly Search

Input: Query $\mathbf{q}$, predicate ϕ, result size k, layer ℓ, entry cube c_0, budget ef
Output: Top-k results satisfying ϕ

1 $R \leftarrow \emptyset$;
2 Initialize a dynamic cube bitmap B with $B[c_0] = 1$ and $B[c] = 0$ for others;
3 Initialize a priority queue Q with the entry point of c_0;
4 **while** $Q \neq \emptyset$ ***and*** *($|R| < k$* ***or*** *the closest candidate in Q is closer than the k-th result in R)* **do**
5 $\quad p \leftarrow$ pop the closest candidate from Q;
6 $\quad$ **for each** *neighbor u of p via an intra- or cross-cube edge* **do**
7 $\quad\quad$ **if** $B[u.\text{cube}] = 1$ ***or*** $\phi(\mathbf{s}_u) = 1$ **then**
8 $\quad\quad\quad$ **if** $B[u.\text{cube}] = 0$ **then** $B[u.\text{cube}] \leftarrow 1$;
9 $\quad\quad\quad$ **if** $\phi(\mathbf{s}_u) = 1$ **then** insert u into R;
10 $\quad\quad\quad$ **if** u has not been visited **then** insert u into Q;
11 $\quad\quad\quad$ Retain only the best ef candidates in Q;
12 **return** the top-k results in R;

located efficiently by binary search. We theoretically justify this layer selection criterion in § 4.3.

Phase 2: Search-Mode Selection. After selecting the layer ℓ^*, CubeGraph performs beam search over the graph induced by the intra-cube and cross-cube edges at that layer. The remaining question is how to identify the cubes that participate in the search. One approach is to materialize all cubes intersecting the query region before traversal; the other is to discover relevant cubes incrementally during traversal. The choice depends on whether the query predicate supports efficient cube–region intersection tests.

Accordingly, CubeGraph provides two search modes: *predetermined search* and *on-the-fly search*. If ϕ supports efficient cube–region intersection tests, Algorithm 1 invokes predetermined search (Lines 3–4), which materializes all searchable cubes before graph traversal. This mode is well suited to simple predicates, such as axis-aligned boxes. Otherwise, Algorithm 1 invokes on-the-fly search (Lines 5–7), which first selects an entry cube c_0 from a representative point of ϕ and then discovers searchable cubes during traversal.

Predetermined Search. Algorithm 2 is used when the cubes intersecting ϕ can be identified efficiently. It first computes the cube cover C of ϕ and records it in a bitmap B (Lines 1–3). The search is then initialized from the entry points of these cubes (Line 4), which are stored in their corresponding local graph indexes. During traversal, each candidate object expands through both intra-cube and cross-cube edges (Lines 5–7). A cross-cube edge is traversed only if its target cube is marked in B (Line 8), thereby confining the search to the precomputed cube cover. Each visited object is further checked against ϕ before being inserted into R (Line 9), since an intersecting cube may contain both predicate valid and invalid objects. Unvisited candidates are inserted into the beam queue, whose size is bounded by the beam width ef (Lines 10–11). Finally, the algorithm returns the top-k valid objects in R (Line 12).

On-the-Fly Search. Algorithm 3 is used when enumerating all intersecting cubes is expensive. Instead, cube intersection tests are performed incrementally as new cubes are reached during graph traversal. It starts from an entry cube c_0 and maintains a dynamic bitmap B of active cubes (Lines 1–3). During traversal, a neighbor u is explored if it belongs to an active cube or satisfies ϕ (Lines 4–6). In the latter case, CubeGraph activates the cube containing u by updating B (Lines 7–8), enabling subsequent traversal within that cube and across its adjacent cubes. As in predetermined search, only objects satisfying ϕ are inserted into R, while unvisited searchable neighbors are added to the beam queue, whose size is bounded by ef (Lines 9–11). Finally, top-k valid objects in R is returned.

Example 4.5. The right-hand panels of Fig. 1 illustrate a rectangle with a circular exclusion region. Predetermined search marks the intersecting cubes in bitmap B before traversal and prunes cross-cube edges whose target cubes are unmarked. On-the-fly search starts from an entry cube and activates another cube when traversal reaches an object satisfying the composite predicate. Within an active cube, predicate-invalid objects may still support routing, while only objects satisfying ϕ enter the result set.

Remark. The two search schemes differ in how searchable cubes are identified. Predetermined search computes the cube cover of the query region before traversal and keeps the bitmap B fixed, making it effective when cube–region intersection tests are inexpensive. In contrast, on-the-fly search starts from a single active cube and activates new cubes only after reaching objects satisfying ϕ. Both schemes identify only the cubes intersecting ϕ, but differ in how they balance cube discovery and graph traversal: predetermined search incurs an upfront cube-enumeration cost, whereas on-the-fly search discovers searchable cubes incrementally during traversal but may perform repeated cube membership checks. *In practice, we use predetermined search for axis-aligned box predicates and on-the-fly search for all other predicate types.* § 6 shows that this hybrid strategy delivers strong performance in practice.

4.3 Analysis

We first bound the number of cubes intersecting a region, then analyze the eligible fraction of their candidate pool, and finally give a conditional query-work bound. These statements separate geometric coverage from actual graph traversal and recall.

Justification of Layer Selection. The normalized attribute space is $\mathcal{S} = [0, 1]^m$. The following cube-count bound is geometric and does not assume any distribution of objects. At layer ℓ, each dimension is partitioned into $g_\ell = 2^{\ell+1}$ intervals, yielding g_ℓ^m cubes, each with width $w_\ell = 1/g_\ell$. For a region predicate ϕ, let $r = \text{len}(\phi)$ denote its characteristic length, defined as the maximum side length of its tight bounding box. Whenever possible, CubeGraph selects the preferred layer ℓ^* satisfying $r/2 < w_{\ell^*} \leq r$. Since adjacent layers differ in cube width by a factor of two, such a layer exists whenever r

lies within the supported resolution range. Otherwise, CubeGraph selects the layer whose cube width is either immediately above r or below $r/2$. For simplicity, the following analysis assumes that the preferred layer exists.

Layer-Selection Guarantee. Recall that existing partition-based methods face a fundamental trade-off. Coarse partitioning produces fewer but larger partitions, preserving graph connectivity while introducing many irrelevant objects into each selected partition. In contrast, fine partitioning yields smaller and more selective partitions but increases graph fragmentation. To balance geometric selectivity and graph connectivity, CubeGraph selects the layer ℓ^* satisfying $r/2 < w_{\ell^*} \leq r$. We first bound the number of cubes intersecting the query region.

Proposition 4.6. *For a region predicate ϕ with characteristic length r, selecting the layer ℓ^* satisfying $r/2 < w_{\ell^*} \leq r$ guarantees that ϕ intersects at most 3^m cubes.*

The proposition bounds cube count, not occupancy or query time. If no preferred layer exists, the general bound is $\prod_i(\lceil a_i/w_\ell \rceil + 1)$ for bounding-box side lengths a_i; the 3^m bound need not hold at the finest available layer. Moreover, removing a hole from a box leaves its characteristic length unchanged. A thin diagonal region can likewise retain a large box. Thus, matching w_ℓ to r controls resolution without optimizing shape fit or adapting partitions to skew.

Elastic Factor. We use the *elastic factor* [45] to characterize the candidate cover. Let C contain all cubes intersecting ϕ, and let $\mathcal{G}^*$ contain all objects in their union $U = \bigcup C$. Then $\mathcal{D}_\phi \subseteq \mathcal{G}^*$. This is a full geometric cover, not the set of vertices visited by a finite beam search. On-the-fly discovery may reach only part of this cover, particularly across empty gaps or disconnected components.

Definition 4.7 (Elastic Factor). The *elastic factor* is

$$e(\mathcal{D}_\phi, \mathcal{G}^*) = \frac{|\mathcal{D}_\phi|}{|\mathcal{G}^*|}.$$

The elastic factor measures candidate-pool selectivity, not the valid fraction along the search path. For independent samples from any fixed joint attribute distribution μ, including correlated coordinates, and a fixed query and partition with $\mu(U) > 0$, the law of large numbers gives

$$e \xrightarrow[N\to\infty]{\text{a.s.}} \frac{\mu(\phi)}{\mu(U)}.$$

Uniform attributes are the special case in which this ratio equals the *volume elastic factor*

$$\bar{e}(\phi, \ell^*) = \frac{\mathrm{Vol}(\phi)}{\mathrm{Vol}(U)}.$$

For skewed data, a geometric volume ratio alone does not bound the empirical eligible fraction. Vector–attribute correlation can further bias graph traversal even when the candidate-pool ratio is large.

Lemma 4.8. *For an m-dimensional ball of diameter r contained in $[0,1]^m$, under the layer-selection rule in Proposition 4.6, the volume elastic factor satisfies*

$$\bar{e} \geq \frac{\pi^{m/2}}{6^m \Gamma(m/2+1)}.$$

This is a fixed-m bound on the full cover; the constant decreases with dimensionality. For general regions, $\bar{e} \geq \mathrm{Vol}(\phi)/(3^m r^m)$, which can be arbitrarily small for thin or hollow shapes. Neither bound ensures that graph traversal visits every eligible object or reaches a specified recall.

Query-Time Bound. We next derive the query complexity of CubeGraph. A query consists of three phases. First, CubeGraph identifies the preferred layer ℓ^*. Second, it discovers the cubes intersecting the query region using either predetermined or on-the-fly search. Finally, it performs beam search on the graph induced by the selected cubes. We use C to denote the startup cost of beam search over the graph index, abstracting away the implementation details of the underlying graph index. The overall query complexity is given below.

Theorem 4.9. *Given a query $(\mathbf{q}, \phi)$ with characteristic length r, let* CubeGraph *select layer ℓ^*. Assume that (1) cube discovery costs T_{cube}; (2) processing each distinct candidate, including distance, predicate, bitmap, and queue operations, costs $O(1)$ for fixed dimensions and search parameters; (3) expected startup work is C; and (4) until k valid candidates are found, traversal can continue and each new candidate is valid with conditional probability at least $c \in (0,1]$. Then the expected time to retrieve k valid candidates is*

$$O(\log L + T_{\text{cube}} + C + k/c).$$

Interpretation. The merged graph $\mathcal{G}^*$ consists of the selected cubes together with their cross-cube edges. The theorem counts work to obtain k valid candidates under explicit assumptions; it does not bound the work for a target ANN recall. In particular, neither the cover ratio e nor the ball bound establishes the path-dependent probability c. The startup term C depends on graph quality and the workload; logarithmic average search cost is a modeling assumption used for graph indexes [17, 18, 26, 29], not a distribution-free guarantee here.

For predetermined search, Proposition 4.6 gives $T_{\text{cube}} = O(3^m)$ under constant-time cube–predicate intersection tests, whereas on-the-fly search amortizes cube discovery into the $O(1)$ predicate and bitmap checks performed during traversal. Finally, the term k/c captures the filtering overhead: if the conditional validity probability stays bounded below until k candidates are found, $O(k)$ post-startup visits suffice under the stated cost model. Exact membership checking ensures valid outputs; finite-budget graph exploration still determines recall.

5 INDEX CONSTRUCTION AND MAINTENANCE

We first present the construction of the CubeGraph index in § 5.1, and then describe its maintenance under dynamic updates in § 5.2.

5.1 Index Construction

The previous subsection defines CubeGraph as a hierarchy of L layers, where each layer consists of a sparse cube directory, intra-cube graphs, and cross-cube edges (Definition 4.1). We now describe how to construct the index efficiently. For each layer, the construction proceeds in two phases. Phase 1 partitions the attribute space into cubes, assigns each object to its corresponding cube, and builds

Algorithm 4: Hierarchy Construction with Local Indexing

Input: Dataset $\mathcal{D}$, number of layers L, parameters M, ef_c
Output: Local hierarchy $\mathrm{CI}^{\mathrm{Local}}$

1 Normalize attributes and initialize the attribute space $\mathcal{B} = [0,1]^m$;
2 **for** $\ell = 0$ **to** $L-1$ **do**
3 Set granularity $g_\ell = 2^{\ell+1}$ and cube side length $w_\ell = 1/g_\ell$;
4 Partition $\mathcal{B}$ into $(g_\ell)^m$ cubes;
5 **for each** $(\mathbf{x}_i, \mathbf{s}_i) \in \mathcal{D}$ ***in parallel*** **do**
6 Compute the cube ID of object i and assign it to the corresponding cube;
7 **for each** *non-empty cube* c ***in parallel*** **do**
8 Build the local graph index for objects in c using M and ef_c;

Algorithm 5: Cross-Cube Edge Construction

Input: Local hierarchy $\mathrm{CI}^{\mathrm{Local}}$, parameter M_{cross}
Output: CubeGraph index

1 **for each** *non-empty cube* c *at each layer* ℓ **do**
2 **for each** *object* p *in* c ***in parallel*** **do**
3 **for each** *face-adjacent cube* c_{adj} *of* c **do**
4 Perform an AKNN search in c_{adj} using p as the query;
5 Select the M_{cross} nearest objects returned by the search;
6 Add the selected objects as cross-cube neighbors of p;

the local graph $\mathcal{G}_\ell^{\mathrm{intra}}$ within every non-empty cube by creating intra-cube edges. Phase 2 constructs $\mathcal{G}_\ell^{\mathrm{cross}}$ by linking each object to nearby objects in face-adjacent cubes through cross-cube edges. Since cube assignment, local graph construction, and adjacent-cube searches are independent across cubes and objects, both phases are naturally parallelizable. We describe these two phases below.

Phase 1: Local Graph Construction. Algorithm 4 constructs the sparse cube directories and local graph indexes corresponding to C_ℓ and $\mathcal{G}_\ell^{\mathrm{intra}}$ for each layer ℓ. Specifically, given a dataset $\mathcal{D}$, we first normalize each attribute dimension to the unit domain $\mathcal{B} = [0,1]^m$ (Line 1), consistent with § 4.1. For each layer $\ell \in [0, L-1]$, we set $g_\ell = 2^{\ell+1}$ and partition $\mathcal{B}$ into $(g_\ell)^m$ equal-sized cubes with side length $w_\ell = 1/g_\ell$ (Lines 2–4). Each object is then assigned to exactly one cube according to its normalized attribute vector (Lines 5–6). Only non-empty cubes are materialized, and a bounded-degree local graph index is built for the objects in each cube using parameters M and ef_c, following the standard HNSW construction procedure [27] (Lines 7–8). This phase also records the IDs of face-adjacent cubes for Phase 2: An m-dimensional cube has at most $2m$ face-adjacent neighbors, each differing by one grid step along a single dimension.

Phase 2: Cross-Cube Edge Construction. After Phase 1, each layer consists of accurate but disconnected local graphs. Algorithm 5 constructs the cross-cube graph $\mathcal{G}_\ell^{\mathrm{cross}}$, enabling traversal between neighboring cubes to further improve graph connectivity. Specifically, for each layer ℓ, each object p in cube c, and each face-adjacent cube c_{adj}, CubeGraph uses p as the query and performs a standard HNSW beam search over the local graph of c_{adj} (Lines 1–4). It then stores the top-M_{cross} nearest objects as a separate cross-cube neighbor group of p (Lines 5–6). Separating cross-cube and intra-cube neighbors follows the memory layout in Fig. 2, allowing query processing to traverse only the cross-cube neighbor groups whose target cubes satisfy the query predicate.

Example 5.1. Consider a 2D attribute space with $L = 3$ in Fig. 1. Layer 0 partitions the space into $2^2 = 4$ coarse cubes, while Layers 1 and 2 partition it into $4^2 = 16$ and $8^2 = 64$ cubes, respectively. As shown in the upper-left panel of Fig. 1, during Phase 1, each object is assigned to exactly one cube at each layer. Objects within the same non-empty cube are indexed together to form the local graph $\mathcal{G}_\ell^{\mathrm{intra}}$, whereas objects separated in the attribute space belong to different local graphs. As shown in the left-hand panels of Fig. 1, during Phase 2, for each object p in cube c, CubeGraph searches every face-adjacent cube c_{adj} and stores the nearest objects as cross-cube neighbors of p. These edges constitute $\mathcal{G}_\ell^{\mathrm{cross}}$, enabling subsequent query traversal across cube boundaries through explicitly maintained cross-cube neighbor groups.

Index Construction Time. We analyze the overall time complexity of index construction by examining Phase 1 (Algorithm 4) and Phase 2 (Algorithm 5) separately.

Theorem 5.2. *Under an average $O(\log N)$ local insertion/search cost with fixed graph parameters, the two-phase construction takes $O(NLm \log N)$ time.*

Remark. Under the same model, Phase 1 costs $O(NL \log N)$ and Phase 2 costs $O(NLm \log N)$. Cross-cube searches use a small beam (20 in our implementation), whereas the finest-layer construction uses $ef_c = 200$. Their relative wall time depends on cube occupancies and parallelism.

Subgraph reuse. Algorithms 4–5 define a reference construction. The measured builder initializes only the finest-layer local graphs by insertion. A parent inherits child local edges and child cross edges internal to that parent, adds sibling entry points, and performs one synchronous refinement round. Reverse-edge repair precedes cross-cube linking. This reuses local proximity information while refining connections at the coarser resolution. Appendix A.7 gives the fixed-parameter cost model and implementation details [18].

5.2 Index Maintenance

In practice, objects may be inserted into or deleted from the dataset $\mathcal{D}$ after the CubeGraph index has been constructed. A straightforward solution is to rebuild the entire index, but this unnecessarily recomputes the local graphs and cross-cube edges for all objects. Instead, we observe that inserting or deleting a single object affects only a small number of cubes. Therefore, CubeGraph supports incremental maintenance by updating only the cubes affected by the modified object. Below, we describe the index maintenance procedures for object insertion and deletion.

Object Insertion. When a new object $o = (\mathbf{x}, \mathbf{s})$ arrives, CubeGraph inserts it into every layer of the index. For each layer $\ell \in [0, L-1]$, it first computes the cube ID of o from $\mathbf{s}$ using the same assignment rule as Algorithm 4. If the target cube does not exist, a new entry is created in the sparse cube directory; otherwise, the existing cube is reused. The object is then inserted into the cube's local graph using the standard graph-index insertion procedure with parameter M, incrementally updating $\mathcal{G}_\ell^{\mathrm{intra}}$. After the local insertion, CubeGraph constructs the outgoing cross-cube neighbor groups of o. For each non-empty face-adjacent cube, it performs an AKNN search over that cube's local graph and stores the top-M_{cross} results as the corresponding cross-cube neighbor group, following Algorithm 5. Consequently, each insertion introduces at most M

Table 2: Datasets used in the shared-query evaluation.

Dataset	Size	Dim	Queries used	Attributes
SIFT1M	1,000,000	128	1,000	2D/3D/4D Uniform
YFCC	999,894	512	1,000	2D/3D Geolocation
WIKI	982,119	384	1,000	2D/3D Geolocation
MSMARCO10M	10,000,000	1024	1,000	2D/3D/4D Uniform

WIKI lists unique vectors; exact duplicates removed (1,170,381 → 982,119).

intra-cube edges and $2mM_{\text{cross}}$ outgoing cross-cube edges per layer while preserving the bounded-degree index structure.

Object Deletion. Object deletion is handled lazily. Physically removing an object from a graph index requires repairing both local and cross-cube neighbor lists, which is expensive. Instead, CubeGraph simply marks the deleted object as invalid without modifying the graph structure, preserving correctness. Specifically, when an object $o = (\mathbf{x}, \mathbf{s})$ is deleted, CubeGraph locates its record at each layer and marks it as invalid. During query processing, invalid objects are skipped and are not counted toward the top-k results. Since the adjacency lists remain unchanged, the deletion cost is proportional only to the number of hierarchy layers. As with existing graph indexes, excessive invalid records may gradually degrade search performance. In practice, CubeGraph performs local compaction or rebuilds a cube when its invalid-record ratio exceeds a predefined threshold, while routine updates use the lightweight lazy deletion strategy.

Dynamic Update Complexity. We next analyze the time complexity of incremental maintenance.

Theorem 5.3. *Assuming average $O(\log N)$ local insertion/search cost with fixed graph parameters, insertion takes $O(Lm \log N)$ time. Lazy deletion takes $O(L)$ time with an object-to-record mapping.*

6 EXPERIMENTS

We evaluate query efficiency (Exp-1), physical components and search modes (Exp-2), complex predicates (Exp-3), attribute dimensionality (Exp-4), index cost (Exp-5), and maintenance (Exp-6). The technical report includes the complete appendix and workload matrix.[1]

6.1 Experimental Setup

Datasets and attributes. Table 2 summarizes four datasets. SIFT1M [4] contains one million 128D vectors; MSMARCO10M contains ten million 1024D passage embeddings. Both use uniform synthetic attributes in $[0, 1]^m$, $m = 2, 3, 4$. YFCC contains one million 512D CLIP image embeddings, and WIKI contains 982,119 distinct 384D WikiText-103 passage embeddings. Both use real 2D geolocation and a normalized timestamp for 3D. A dataset's "mD" suffix denotes its attribute dimensionality, independently of its vector dimension.

Baselines. We compare with PostFiltering, which searches an unfiltered HNSW graph before applying the predicate; ACORN-γ [29], with $\gamma = 12$; iRangeGraph [43], using the original implementation's native two-attribute path with external input conversion; and original-author KHI, which partitions attributes with a KD-tree. The evaluated iRangeGraph implementation supports only 2D. KHI and iRangeGraph participate in box comparisons; the complex-predicate comparison uses ACORN and POST with identical membership tests.

[1] https://github.com/mingyu-hkustgz/CubeGraph/blob/main/technical_report.pdf

Table 3: Predetermined/on-the-fly QPS at the common 99% recall floor. Ratios above one favor predetermined.

Dataset	1%	2%	5%	10%
SIFT1M 2D	1.047×	1.076×	1.094×	1.069×
MSMARCO10M 2D	0.997×	0.998×	1.005×	1.001×

Workloads and metrics. Box queries have 1/2/5/10% qualifying fractions, measured from actual object counts. Each method receives the same 1,000 queries and exact ground truth ($k = 20$). Recall@k is $|R \cap A|/k$, with exact result R and returned result A; efficiency is QPS. Curves show median recall and QPS over three repeats. Quoted operating points use each method's smallest width achieving at least 99% recall in *every* repeat; N/R means no tested width qualifies. Widths range from 20 to 2048, extended to 4096 for ACORN on MSMARCO10M 3D/4D (Appendix A.9). Complex predicates use frozen, count-calibrated workloads (Exp-3).

Implementation and execution. All methods run on the same machine with 1.5 TiB RAM, with vectors and indexes in memory and single-threaded queries. CubeGraph builds with 128 threads: finest-layer graphs use $M = 16$, $ef_c = 200$; parent layers reuse child graphs and perform one beam-64 refinement; cross-edge searches use beam 20 and retain two edges per occupied face-adjacent cube. Box comparisons use predetermined search; complex predicates and physical ablations use on-the-fly search. Exp-2 compares the modes on identical indexes. Appendices A.7–A.8 give implementation, auditing, and timing scopes.

6.2 Experimental Results

Exp-1: Overall query efficiency. CubeGraph is particularly effective for selective multi-attribute predicates. At 1%, it outperforms KHI on *all six* 3D/4D configurations by 1.41–3.89×: SIFT 3D/4D achieve 2.23/3.89×, YFCC 3D 2.28×, WIKI 3D 1.41×, and MSMARCO10M 3D/4D 2.07/2.17×. On real YFCC 3D, the advantage persists across all four fractions (1.30–2.28×). Overall, CubeGraph leads at 19 of 24 higher-dimensional operating points.

Figures 3–4 show 2D–4D cases, and Appendix A.3 retains every additional 2D/3D curve. On SIFT 2D, the CubeGraph/KHI ratios are 1.86/1.46/0.94/0.75× at 1/2/5/10%; WIKI 2D gives 1.20/1.28/0.73/0.70×. Thus, the benefit depends on selectivity: KHI can lead for broader filters and leads on YFCC/MSMARCO10M 2D. POST does not reach 99% recall at 1% on any of the ten configurations, although it is competitive for broader predicates.

Exp-2: Components and search modes. We build four structures on SIFT 2D/3D and YFCC 2D: the hierarchy and a resolution-matched finest-layer index, each with and without cross edges. All variants use on-the-fly search. At the 99% recall floor, the hierarchy/single-layer throughput ratios span 1.58–5.42× on SIFT 2D and 0.94–5.02× on SIFT 3D where both reach the floor. Disabling cross edges limits the hierarchy to maximum recall 0.522/0.476/0.681 on the three configurations. On SIFT 2D, the hierarchy uses 5.8× the

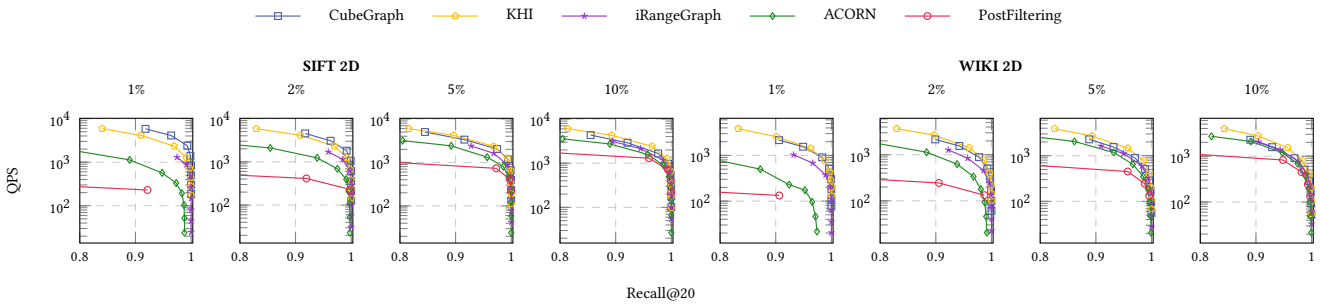


Figure 3: 2D recall–QPS curves on 1,000 shared queries ($k = 20$): medians of three repeats. Each block varies the qualifying fraction from 1% to 10%. Appendix A.3 contains YFCC and MSMARCO10M 2D; all measured settings are retained.

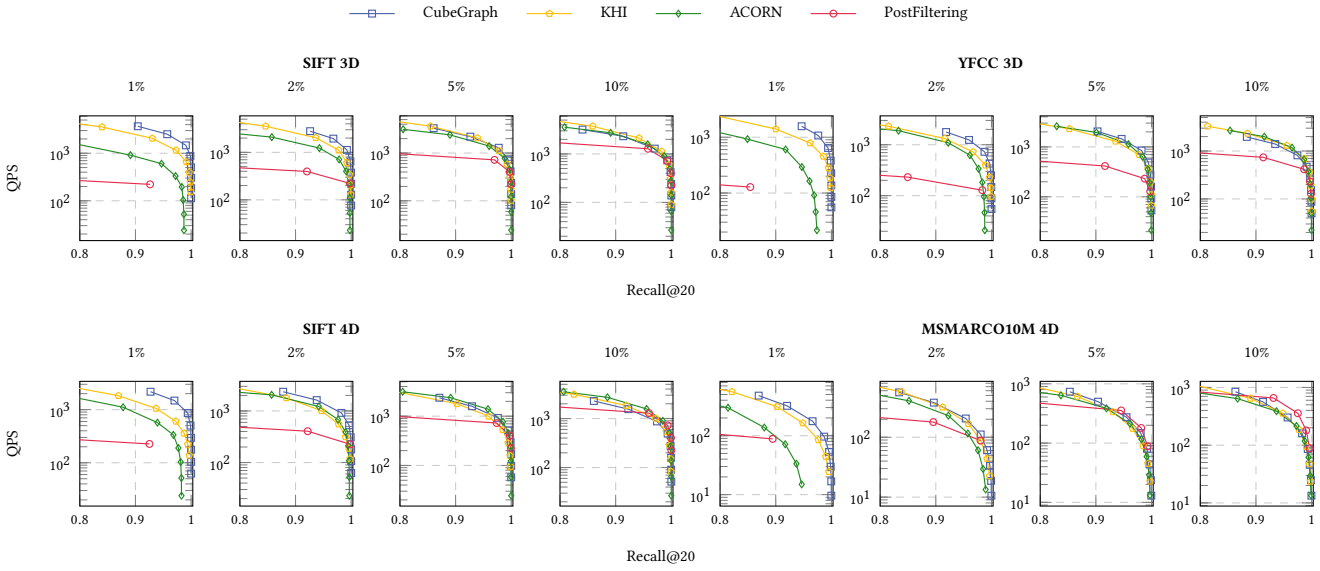


Figure 4: 3D/4D box filters on SIFT1M, real YFCC attributes, and MSMARCO10M. Each block includes all four qualifying fractions. Appendix A.3 gives the remaining 3D configurations; iRangeGraph has no evaluated 3D/4D path.

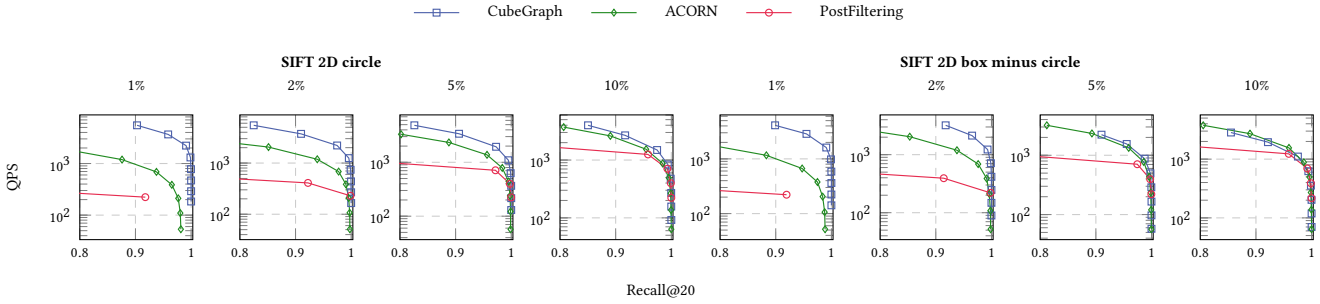


Figure 5: Complex predicates on SIFT1M: circle and box-minus-circle, each at 1/2/5/10% qualifying fractions. All methods share frozen predicates and exact ground truth. Three-repeat median recall–QPS curves; timing excludes predicate preparation. Appendix A.4 includes 3D/4D balls and preparation costs.

serialized space and 5.6× the build time of the single layer. Appendix A.2 reports all variants, independent builds and search counters.

Predetermined versus on-the-fly. On identical six-layer indexes and box workloads, predetermined search gives 1.047–1.094× the on-the-fly throughput on SIFT1M 2D; MSMARCO10M 2D is nearly tied (0.997–1.005×). Both modes select the same qualifying widths. Table 3 gives all eight ratios; Appendix A.11 reports absolute QPS, full curves, and the separate-pass timing protocol.

Table 4: SIFT1M attribute-dimension probes: native construction time and serialized size excluding source vectors.

Attributes m	Layers L	Build (s)	Size (MB)
2	6	65.2	1088
3	4	52.0	812
4	3	41.7	676
6	2	37.3	544
8	1	26.7	332

Table 5: MSMARCO10M 2D: native construction time and serialized space, excluding source vectors. Ratios are method/CG; build settings differ (Appendix A.8).

Method	Layers	Time (s)	/CG	Size (GB)	/CG
CG	6	4,328.6	1.00×	10.880	1.00×
KHI	25	3,575.8	0.83×	19.468	1.79×
iRG	25	19,366.6	4.47×	18.420	1.69×

Table 6: Insert-only updates: seconds per batch (32 threads, one run). Ratio = rebuild/incremental time.

	SIFT1M			YFCC		
Batch	Inc.	Rebuild	Ratio	Inc.	Rebuild	Ratio
10%	0.78	4.66	5.96×	1.97	12.14	6.18×
30%	2.04	5.39	2.64×	5.64	15.77	2.80×
50%	3.74	7.07	1.89×	9.75	20.19	2.07×

Exp-3: Complex region predicates. Figure 5 evaluates 2D circles and box-minus-circle predicates. At 1%, only CubeGraph reaches the common recall floor, at 2,221 and 1,612 QPS respectively. At 2%, it is 3.25/3.16× faster than ACORN and 5.45/5.59× faster than POST. The benefit extends to 3D/4D balls at 1%, where only CubeGraph reaches the floor (Appendix A.4). These comparisons demonstrate selective non-box filtering; they do not imply a speedup over KHI, whose evaluated path handles boxes.

All four fractions are retained: POST overtakes CubeGraph for the 10% box-minus-circle and 3D/4D ball predicates. Methods share exact predicates and ground truth within each family. QPS excludes predicate/mask preparation, reported separately in A.4. Equal qualifying counts across families do not isolate shape cost, and these on-the-fly results are not a search-mode ablation.

Exp-4: High-dimensional attribute probes. As attributes increase from 2 to 8, the occupancy rule (at least 50 objects per cube on average) reduces retained depth from six layers to one. Table 4 reports build time and space; Appendix A.6 gives occupancies. At 8D, multi-layer routing is unavailable. The 6D/8D probes concern construction; query quality is evaluated up to 4D.

Exp-5: Index construction and storage. CubeGraph bounds replication by L, giving $O(NLm)$ space. On MSMARCO10M 2D, six layers occupy 10.880 GB, 44.1%/40.9% below KHI/iRangeGraph with 25 levels (Table 5). The table includes native construction times and method/CG cost ratios; build settings and timing scopes differ (A.8). Retaining five layers saves 16.5% space on SIFT1M/MSMARCO10M 2D and returns identical IDs at all 96 settings per dataset (A.9). These variants preserve the finest resolution and drop the coarsest layer; the equivalence is workload-specific, and depth alone does not explain all byte differences.

Exp-6: Dynamic maintenance. With 32 construction threads, we insert 10/30/50% batches into 100K initial vectors from a 200K pool. Incremental maintenance is 1.89–6.18× faster than rebuilding the same resulting dataset (Table 6); its benefit decreases with batch size. Appendix A.5 reports post-update recall and QPS at the measured points closest to 95% recall, where incremental QPS does not always exceed rebuilding. These insert-only tests use a 100K-object initial index; they do not establish sustained mixed-update performance at the full dataset scale.

7 RELATED WORK

§ 2.2 reviewed the foundations of AKNN search and index merging. Here, we focus on attribute-constrained AKNN search, which is most closely related to region-constrained AKNN search. Existing studies can be broadly categorized into numeric-constrained and label-constrained AKNN search.

Numeric-constrained AKNN Search. Numeric-constrained AKNN methods retrieve vectors satisfying numeric predicates [16, 23, 31, 39, 44, 45, 50, 53]. Most focus on one-dimensional range predicates by compressing interval representations. SeRF [53] compresses the $O(N^2)$ interval space into a compact structure, while segment-tree-based methods [8, 40, 43] build hierarchical graph indexes over one-dimensional ranges. Hi-PNG [44] also adopts a hierarchical design but studies interval-filtered ANNS, where both data and query vectors are associated with numerical intervals. Although effective for one-dimensional ranges and interval predicates, these methods do not naturally support arbitrary multi-dimensional geometric predicates. CubeGraph instead uses a selected grid resolution and stored face-adjacent links, with exact membership checks for numeric region predicates.

Label-constrained AKNN Search. For categorical attributes such as labels, existing methods build or combine indexes based on label categories [13, 22, 25, 35, 42, 47, 51]. UNG [7] connects per-label graphs through cross-label edges, while UniFilter [42] constructs an automaton-guided navigation graph on top of a base index. These methods are designed for exact label predicates rather than continuous geometric regions. CubeGraph uses selected cubes as a geometric candidate cover, while cross-cube links support traversal within its reachable components.

8 CONCLUSION

This paper presents CubeGraph, a hierarchical graph index for region-constrained AKNN search. CubeGraph combines a bounded-depth multi-resolution cube hierarchy with cross-cube edges to control geometric selectivity and enable cross-partition traversal. We propose a principled layer-selection strategy, along with predetermined and on-the-fly search algorithms for efficient query processing. We also develop efficient index construction and maintenance algorithms. Extensive experiments on multiple datasets demonstrate that CubeGraph supports diverse predicate types, with a workload-dependent recall–throughput trade-off. Future work includes distributed implementations, occupancy-aware layer selection, and controlled comparisons that vary skew and region fill ratio. These studies will quantify when finer geometric pruning offsets the additional index and traversal costs.

REFERENCES

[1] 2026. Zekai Wu, Jiabao Jin, Peng Cheng, Xiaoyao Zhong, Lei Chen, Yongxin Tong, Zhitao Shen, Jingkuan Song, Heng Tao Shen, Xuemin Lin. *arXiv preprint arXiv:2603.21710* (2026). https://arxiv.org/abs/2603.21710

[2] Md Mahbub Alam, Luis Torgo, and Albert Bifet. 2022. A survey on spatio-temporal data analytics systems. *Comput. Surveys* 54, 10s (2022), 1–38.

[3] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment* 9, 4 (2015).

[4] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.

[5] Artem Babenko and Victor Lempitsky. 2014. Additive quantization for extreme vector compression. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 931–938.

[6] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the 1990 ACM SIGMOD international conference on Management of data*. 322–331.

[7] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–27.

[8] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. *ICML 2024* (2024).

[9] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[10] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[11] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 4 (2014), 744–755.

[12] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.

[13] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.

[14] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.

[15] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems* 32 (2019).

[16] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[17] Chenzhe Jin, Yunan Zhang, Jiayi Liu, and Jianguo Wang. 2026. Efficient Vector Index Merging in Vector Databases. 4, 1 (2026). https://doi.org/10.1145/3786645

[18] Liuchang Jing, Mingyu Yang, Lei Li, Jianbin Qin, and Wei Wang. 2026. Multiple Index Merge for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2602.17099* (2026).

[19] Leonardo Kuffo, Elena Krippner, and Peter Boncz. 2025. PDX: A data layout for vector similarity search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[20] Binhong Li, Xiao Yan, and Shangqi Lu. 2025. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2510.05975* (2025).

[21] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2025. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[22] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736.

[23] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* 18, 4 (Dec. 2024), 1118–1130. https://doi.org/10.14778/3717755.3717770

[24] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: Hierarchical Graph Structure Based on Voronoi Diagrams for Solving Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 15, 2 (2021), 246–258.

[25] Jiarui Luo, Miao Qiao, Chaoji Zuo, and Dong Deng. 2025. Tag-Filtered Approximate Nearest Neighbor Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3642–3654.

[26] Yu A Malkov and Dmitry A Yashunin. [n.d.]. hnswlib. https://github.com/nmslib/hnswlib.

[27] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[28] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Companion of the 2024 International Conference on Management of Data*. 597–604.

[29] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[30] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1 (2023), 54:1–54:27. https://doi.org/10.1145/3588908

[31] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (June 2025), 3256–3268. https://doi.org/10.14778/3748191.3748193

[32] Parikshit Ram and Kaushik Sinha. 2019. Revisiting kd-tree for nearest neighbor search. In *Proceedings of the 25th acm sigkdd international conference on knowledge discovery & data mining*. 1378–1388.

[33] Yitong Song, Bin Yao, Zhida Chen, Xin Yang, Jiong Xie, Feifei Li, and Mengshi Chen. 2025. Efficient top-k spatial-range-constrained approximate nearest neighbor search on geo-tagged high-dimensional vectors. *The VLDB Journal* 34, 1 (2025), 14.

[34] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.

[35] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2022. Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints. *arXiv preprint arXiv:2203.13601* (2022).

[36] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *arXiv preprint arXiv:2502.18113* (2025).

[37] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1 (2024), V2mod014:1–V2mod014:27. https://doi.org/10.1145/3639269

[38] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.

[39] Yuxiang Wang, Ziyuan He, Yongxin Tong, Zimu Zhou, and Yiman Zhong. 2025. Timestamp Approximate Nearest Neighbor Search over High-Dimensional Vector Data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3043–3055.

[40] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–27.

[41] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[42] Jiadong Xie, Jeffrey Xu Yu, Siyi Teng, and Yingfan Liu. 2025. Beyond Vector Search: Querying With and Without Predicates. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–26.

[43] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *arXiv preprint arXiv:2409.02571* (2024).

[44] Ming Yang, Yuzheng Cai, and Weiguo Zheng. 2025. Hi-PNG: Efficient Interval-Filtering ANNS via Hierarchical Interval Partition Navigating Graph. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3518–3529.

[45] Mingyu Yang, Wentao Li, Zhitao Shen, Chuan Xiao, and Wei Wang. 2025. ESG: Elastic Graphs for Range-Filtering Approximate k-Nearest Neighbor Search. *arXiv preprint arXiv:2504.04018* (2025).

[46] Mingyu Yang, Wentao Li, and Wei Wang. 2024. Fast High-dimensional Approximate Nearest Neighbor Search with Efficient Index Time and Space. *arXiv preprint arXiv:2411.06158* (2024).

[47] Mingyu Yang, Wenxuan Xia, Wentao Li, Raymond Chi-Wing Wong, and Wei Wang. 2025. Elastic Index Select for Label-Hybrid Search in Vector Database. *arXiv preprint arXiv:2505.03212* (2025).

[48] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph.

Proceedings of the ACM on Management of Data 3, 1 (2025), 1–28.

[49] Yuanhang Yu, Dawei Cheng, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2026. Efficient Approximate Nearest Neighbor Search under Multi-Attribute Range Filter. *arXiv preprint arXiv:2602.15488* (2026).

[50] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[51] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 377–395.

[52] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, et al. 2025. VSAG: An Optimized Search Framework for Graph-based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2503.17911* (2025).

[53] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.

CubeGraph: Supplementary Appendix

Mingyu Yang
HKUST (GZ) & HKUST
China
myang250@connect.hkust-gz.edu.cn

Wentao Li
University of Leicester
United Kingdom
wl226@leicester.ac.uk

Wenxuan Xia
HKUST (GZ)
China
wxia248@connect.hkust-gz.edu.cn

Wei Wang
HKUST (GZ) & HKUST
China
weiwcs@ust.hk

SUPPLEMENTARY APPENDIX

A.1 Proofs

Proof of Theorem 4.4. Each object appears in exactly one cube at each of the L layers. At each layer, its record stores $O(M) = O(1)$ intra-cube edges and $O(mM_{\text{cross}}) = O(m)$ cross-cube edges, since an m-dimensional cube has at most $2m$ face-adjacent cubes and each adjacent cube maintains at most M_{cross} edges. The object identifier requires $O(1)$ space, while the attribute vector requires $O(m)$ space. Thus, each record occupies $O(m)$ space, yielding a total record size of $O(LNm)$. The sparse cube directory stores one entry for each non-empty cube. Because each object belongs to at most one cube per layer, the number of non-empty cubes is bounded by $O(LN)$. Therefore, the directory requires $O(LN)$ space, and the total index size is $O(LNm + LN) = O(LN(m+1)) = O(LNm)$.

Proof of Proposition 4.6. Let $B(\phi)$ denote the tight axis-aligned bounding box of ϕ, whose side lengths satisfy $a_i \le r$ for all dimensions. Since every cube intersecting ϕ also intersects $B(\phi)$, it suffices to bound the cube count for $B(\phi)$. At layer ℓ^*, a segment of length a_i intersects at most $\lceil a_i / w_{\ell^*} \rceil + 1$ grid intervals. Because $a_i / w_{\ell^*} < 2$, each dimension intersects at most three intervals. Therefore, $B(\phi)$, and hence ϕ, intersects at most $\prod_{i=1}^{m} 3 = 3^m$ cubes.

Proof of Lemma 4.8. By Proposition 4.6, the predicate intersects at most 3^m cubes. Since each cube has volume $w_{\ell^*}^m$ and $w_{\ell^*} \le r$, the searched volume is at most $3^m r^m$. An m-dimensional ball of diameter r has volume

$$\frac{\pi^{m/2}(r/2)^m}{\Gamma(m/2+1)}.$$

Hence,

$$\bar{e} \ge \frac{\pi^{m/2}(r/2)^m}{\Gamma(m/2+1)\cdot 3^m r^m} = \frac{\pi^{m/2}}{6^m \Gamma(m/2+1)}.$$

The volume formula assumes that the ball lies inside the normalized domain. For a boundary-clipped ball, the numerator must use its clipped volume instead. For fixed ϕ and U, the indicators of membership in these sets satisfy the law of large numbers under independent sampling from a joint distribution μ. Dividing their limits gives $e \to \mu(\phi)/\mu(U)$ when $\mu(U) > 0$; uniform sampling yields $\bar{e}$. No independence between attribute coordinates is needed for the probability-mass statement.

Proof of Theorem 4.9. The query consists of three phases. First, CubeGraph identifies ℓ^* by binary search over the L layers using the characteristic length r, requiring $O(\log L)$ time. Second, it discovers the cubes at layer ℓ^*, incurring a cost of T_{cube}. For predetermined search, Proposition 4.6 implies $T_{\text{cube}} = O(3^m)$ under constant-time cube–predicate intersection tests. For on-the-fly search, only the entry cube is initialized explicitly, while subsequent cube activations are charged to the $O(1)$ predicate and bitmap checks performed during traversal, yielding the same asymptotic complexity. Finally, CubeGraph performs beam search on the merged graph $\mathcal{G}^*$. By assumption, reaching the relevant search region incurs an expected cost of C. Until k valid candidates have been found, assume traversal can continue and every new candidate is valid with conditional probability at least c, given its history. The chance of needing more than t visits for the next valid candidate is then at most $(1-c)^t$, so its conditional expected waiting time is at most $1/c$. Summing the waiting times gives at most k/c expected visits. The theorem additionally assumes bounded processing cost per candidate; a variable-width queue or nonconstant predicate test must be charged explicitly. This argument concerns valid candidates, not their nearest-neighbor recall. Summing the three phases gives $O(\log L + T_{\text{cube}} + C + k/c)$.

Proof of Theorem 5.2. Construction consists of two phases: (1) building local graph indexes within non-empty cubes, and (2) constructing cross-cube edges. In Phase 1, each object is inserted into one local graph index at each layer. Using a graph index such as HNSW, each insertion takes $O(\log N)$ time on average as an upper bound on the per-cube insertion cost, resulting in a total cost of $O(NL \log N)$. In Phase 2, each object performs searches over at most $2m$ face-adjacent cubes at every layer. Each search takes $O(\log N)$ time on average as an upper bound on the adjacent-cube search cost, giving a total cost of $O(NLm \log N)$. Since Phase 2 dominates, the overall construction time is $O(NLm \log N)$.

Proof of Theorem 5.3. For insertion, the new object is processed once at each of the L layers. Computing its cube ID and updating the sparse cube directory require $O(m)$ time. Inserting the object into the local graph costs $O(\log N)$ on average. The object then performs AKNN search over at most $2m$ face-adjacent cubes to construct its outgoing cross-cube neighbor groups. Each search costs $O(\log N)$ on average, resulting in $O(m \log N)$ cross-cube maintenance per layer. Thus, the total insertion complexity is $O(L(m + \log N + m \log N)) = O(Lm \log N)$. For deletion, CubeGraph locates the object's record at each layer and marks it as invalid. With the maintained object-to-cube mapping, this requires $O(1)$ time per layer, yielding a cost of $O(L)$.

A.2 Detailed Physical-Structure Ablation

Exp-2: Physical-structure ablation. We isolate hierarchy retention and cross edges with four separately constructed physical indexes on SIFT 2D/3D and YFCC 2D. The single-layer variants use the same finest cube resolution as the hierarchy. Every structure is built three times with 128 requested builder threads; costs use their medians. Parallel construction need not produce identical graphs.

Table 1: Physical-structure ablation on SIFT 2D. Build time and process peak RSS are medians of three independent builds. MB/GB use decimal units.

Structure	Layers	Build (s)	Index (MB)	RSS (GB)
Hierarchy, cross edges	6	65.6	1088	3.75
Hierarchy, no cross edges	6	21.4	896	3.54
Single finest, cross edges	1	11.6	188	1.09
Single finest, no cross	1	5.7	156	1.06

Table 2: Maximum recall over the measured sweep and QPS at the 99%-in-every-repeat floor. "— denotes an unreached floor. Curves query the first build, with three query repeats.

Structure	Max recall S2D/S3D/Y2D	SIFT 2D QPS 1%	5%	10%
Hierarchy, cross edges	1.000/1.000/1.000	2,297	1,112	905
Hierarchy, no cross edges	0.522/0.476/0.681	—	—	—
Single finest, cross edges	1.000/1.000/0.977	1,456	327	167
Single finest, no cross	0.024/0.026/0.602	—	—	—

Each query curve uses the first build and three single-threaded repeats of the on-the-fly path. Counter collection runs separately from timing, so counters do not perturb the reported QPS.

At the 99% recall floor, the hierarchy/single-layer throughput ratios span 1.58–5.42× on SIFT 2D and 0.94–5.02× on SIFT 3D where both reach the floor. Disabling cross edges limits the hierarchy to maximum recall 0.522/0.476/0.681 on the three configurations. On SIFT 2D, the hierarchy uses 5.8× the serialized space and 5.6× the build time of the single layer.

At $ef = 2048$, the SIFT 2D hierarchy selects average layer 2.93 at 1% and 1.00 at 10%. At 1%, distance computations per query are 10,965 with cross edges and 3,355 without them. Counter averages include the 20 warm-up queries preceding each 1,000-query pass. Lower traversal work in the disconnected variant must be interpreted together with its measured recall.

A.3 Additional Experimental Results

Additional 2D comparisons. Figure 1 complements the main-paper SIFT and WIKI panels. At 1/2/5/10% qualifying fractions, the CubeGraph/KHI throughput ratios are 0.68/0.84/0.59/0.74× on YFCC and 0.79/0.68/0.84/0.77× on MSMARCO10M. These cases favor KHI at the common recall floor.

Effect of attribute dimensionality. The main-paper curves cover 2D–4D numeric predicates. On SIFT, the CubeGraph/KHI throughput ratios at 1% are 1.86/2.23/3.89× for 2D/3D/4D; at 10% they are 0.75/1.24/0.92×. Thus, dimensionality and predicate selectivity jointly determine the relative ordering. Every curve uses the same 99%-recall operating-point rule. Section A.6 reports index construction beyond four attributes.

Effect of attribute distributions. SIFT1M uses Uniform, Normal, Clustered, Skewed and Hollow numeric attributes. All five configurations use on-the-fly search on their shared fixed-cardinality workloads. At the 99%-in-every-repeat floor, non-uniform/uniform throughput ratios range from 0.54× to 1.38× over jointly qualifying points. All 16 comparisons reach the floor. The measured effect depends on distribution and selectivity; the plotted attribute shapes do not imply a uniform throughput benefit.

A.4 Complex region predicates

Balls and inclusion–exclusion predicates. We evaluate closed attribute-space balls in $m = 2, 3, 4$ and *inside-box AND outside-circle* in 2D on SIFT1M. The 16 configurations combine these four predicate settings with target qualifying fractions 1/2/5/10%. Each uses the first 1,000 released query vectors, $k = 20$, the native indexes, and widths 20, 32, 64, 128, 256, 512, 1024, and 2048, with three single-threaded repeats without explicit CPU affinity. CubeGraph uses its native on-the-fly radius or composite search; ACORN uses its native per-query boolean mask; PostFiltering retrieves a candidate budget and exactly filters the candidates. The range baselines remain outside this interface coverage; no bounding-box proxy is used. Figure 4 contains the full matrix, including all selectivities for every method. Table 3 applies the same 99%-in-every-repeat rule as the main comparison.

Predicate calibration and exact reference sets. The original uniform attributes are used directly in $[0, 1]^m$. Ball centers are seeded throughout that domain, including positions whose balls are clipped by its boundary. The radius is calibrated from the target-th smallest attribute squared distance; the closed-ball boundary and all ties are retained. For composition, a concentric square is calibrated to contain approximately $N\rho/0.7$ objects, then a circular exclusion removes approximately 30% of those objects, yielding the requested final fraction ρ without a degenerate exclusion region. Box boundaries are included and the closed circle is excluded. Float32 predicate parameters, actual per-query counts, and exact reference IDs are frozen and shared by all methods; no ID-based truncation changes the region. Actual counts differ from the requested counts by at most one object. Depending on dimension and selectivity, 221–993 of the 1,000 balls intersect the domain boundary. An independent NumPy implementation exhaustively checks all 16,000 qualifying sets and filtered top-20 reference sets. Equal embedding distances are ordered by original object ID.

Timing and output audit. Search QPS includes traversal, predicate evaluation during search, and returned-ID materialization; it excludes index loading, mask/predicate preparation, reference construction, validation, and file writes. All three methods use this same timing boundary. Preparation is measured separately (Table 4); a precomputed mask is not free, so search-only QPS is not an end-to-end claim. Returned IDs for every width and repeat are retained and independently checked for uniqueness, predicate satisfaction, and recall against the frozen reference.

Small-set boundary limitation. Separate empty-set and one-eligible-object fixtures exposed out-of-mask IDs in the unmodified ACORN search implementation. These are retained as failed robustness tests, not reported as recall-floor failures or successful tests. CubeGraph and PostFiltering produced only eligible IDs in those fixtures (PostFiltering may miss the lone eligible object). The formal matrix contains approximately 10,000 or more eligible objects per query and is audited separately; it does not establish correctness for empty or underfilled qualification sets.

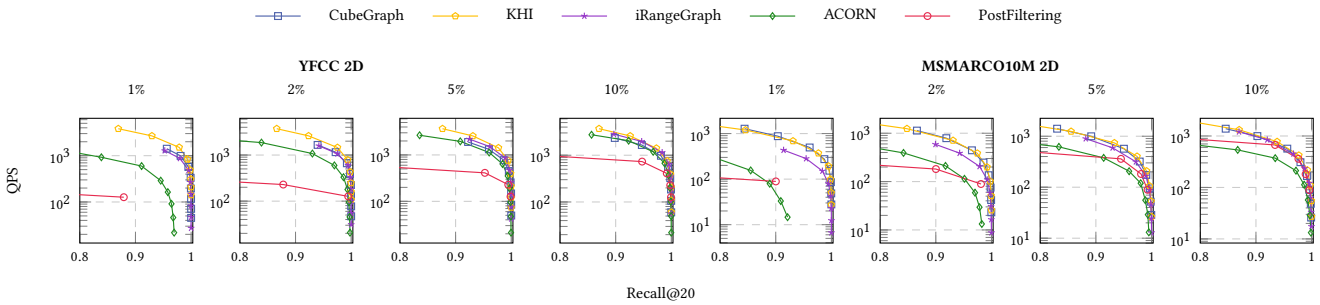


Figure 1: Additional 2D recall–QPS curves, using the same workloads and protocol as the main paper. All methods and four qualifying fractions are retained, including configurations where CubeGraph trails KHI.

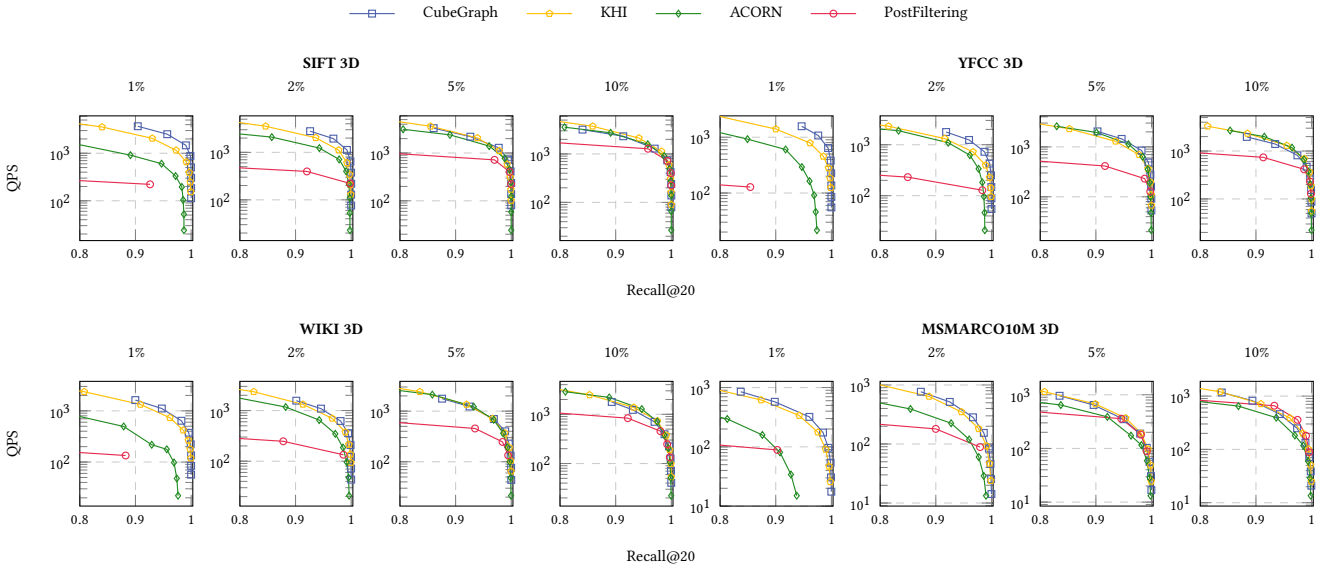


Figure 2: Complete 3D box-filter comparisons, with all four qualifying fractions on each dataset. Medians of three runs on 1,000 shared queries ($k = 20$); iRangeGraph has no evaluated 3D path.

Table 3: Radius/composition: median QPS at the smallest width reaching 99% recall in all three repeats; widths appear in brackets. N/R is followed by maximum measured recall.

Predicate	Method	1%	2%	5%	10%
2D ball	CubeGraph	2,221 [64]	1,249 [128]	1,092 [128]	837 [128]
2D ball	ACORN	N/R (0.982)	384 [256]	440 [256]	492 [256]
2D ball	PostFiltering	N/R (0.918)	229 [2048]	398 [1024]	707 [512]
3D ball	CubeGraph	980 [128]	1,004 [128]	682 [128]	459 [256]
3D ball	ACORN	N/R (0.989)	385 [256]	434 [256]	488 [256]
3D ball	PostFiltering	N/R (0.926)	224 [2048]	393 [1024]	710 [512]
4D ball	CubeGraph	495 [256]	611 [128]	393 [256]	208 [512]
4D ball	ACORN	N/R (0.986)	211 [512]	435 [256]	490 [256]
4D ball	PostFiltering	N/R (0.915)	224 [2048]	404 [1024]	706 [512]
2D box minus circle	CubeGraph	1,612 [64]	1,227 [64]	495 [128]	623 [128]
2D box minus circle	ACORN	N/R (0.989)	388 [256]	432 [256]	496 [256]
2D box minus circle	PostFiltering	N/R (0.919)	220 [2048]	395 [1024]	695 [512]

A.5 Dynamic maintenance

We use a 200,000-vector pool, 100,000 initial vectors and 32 construction threads. Insert-only batches contain 10/30/50% of the initial cardinality. Table 5 reports the wall-clock cost of incremental maintenance and a full rebuild of the same resulting dataset. The incremental/rebuild comparison is internal to this update experiment. The rebuild/incremental time ratios range from 1.89× to 6.18×. Table 6 gives the measured setting closest to 95% recall for

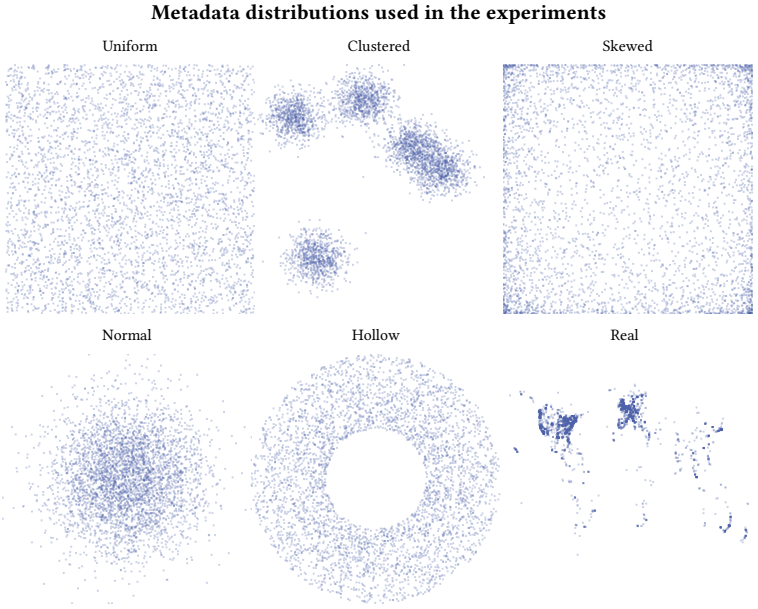


Figure 3: The five synthetic attribute distributions used for the robustness experiment plus the real geolocation metadata of YFCC, each subsampled to 4000 of the 1,000,000 objects for rendering as native vector scatter. Coordinates are normalised per panel; the Real panel is therefore shape-accurate but not a geographic map.

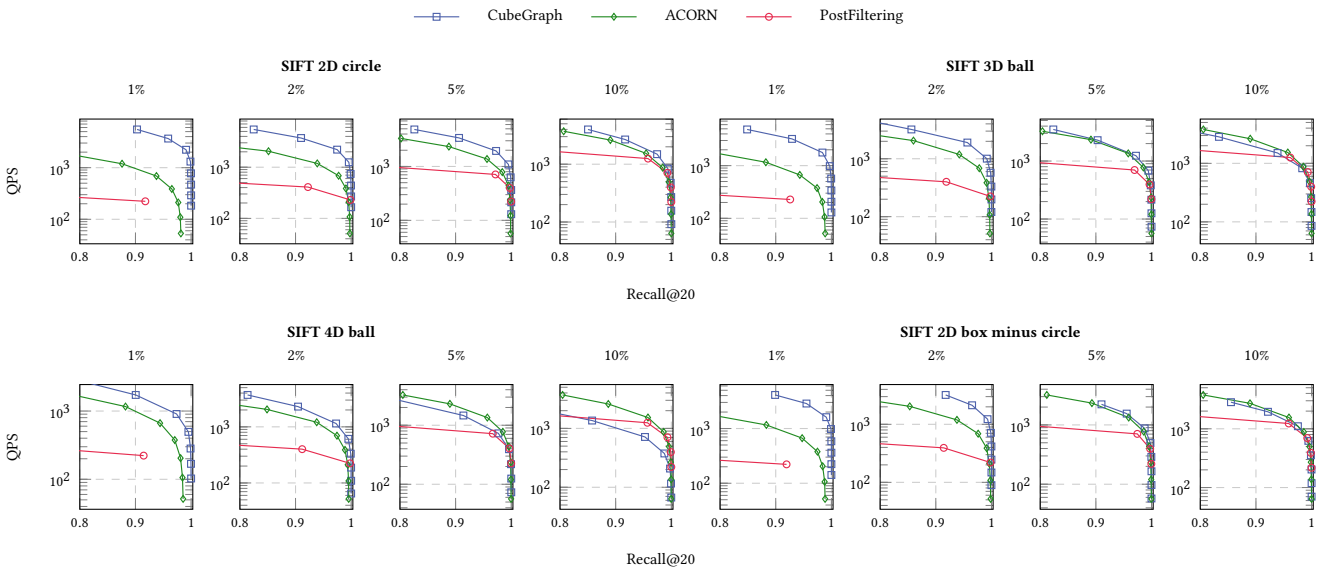


Figure 4: Complete radius and composition matrix: four target qualifying fractions, three methods, eight widths, and three repeats. All curves use the same frozen predicates and exact ground truth within each panel. QPS excludes preparation; Table 4 reports that cost.

each index after the batch. Search is single-threaded and timed with a wall clock; returned IDs are checked for uniqueness and predicate membership outside timing. The table reports each achieved recall explicitly: these are not common-99%-floor operating points.

A.6 High-dimensional attribute probes

The adaptive rule stops adding layers when average occupancy would fall below 50 points per cube. The following SIFT1M builds measure the resulting geometry and native construction cost. Source vectors are excluded from serialized size.

Table 4: ACORN serial mask preparation, milliseconds per query, averaged over the 1,000 queries in one preparation pass per configuration. CubeGraph prepares a constant-size predicate object; POST evaluates the predicate during its timed search.

Predicate	1%	2%	5%	10%
2D ball	2.81	2.78	2.82	2.80
3D ball	3.30	3.25	3.23	3.25
4D ball	3.02	3.06	3.01	3.04
2D box minus circle	8.68	9.39	10.40	10.99

Table 5: Insert-only dynamic maintenance: seconds per batch, one measurement per configuration, 32 construction threads.

	SIFT1M			YFCC		
Batch	Inc.	Rebuild	Ratio	Inc.	Rebuild	Ratio
10%	0.78	4.66	5.96×	1.97	12.14	6.18×
30%	2.04	5.39	2.64×	5.64	15.77	2.80×
50%	3.74	7.07	1.89×	9.75	20.19	2.07×

Table 6: Post-update search quality and QPS at each index's measured setting closest to 95% recall.

	Incremental		Rebuild	
Batch	Recall	QPS	Recall	QPS
SIFT				
10%	93.2%	7,760	93.1%	7,041
30%	93.2%	7,237	92.8%	7,160
50%	93.1%	5,698	92.7%	6,215
YFCC				
10%	95.9%	1,245	95.9%	1,288
30%	95.7%	1,464	95.9%	1,537
50%	95.6%	1,387	95.8%	1,454

m	Layers	Finest cubes	Points/cube	Build (s)	MB
2	6	4096	244	65.2	1088
3	4	4096	244	52.0	812
4	3	4096	244	41.7	676
6	2	4096	244	37.3	544
8	1	256	3906	26.7	332

At eight attributes the rule retains one layer, so multi-layer routing is unavailable. These 6D/8D probes measure construction only; no query-quality claim beyond four attributes is made.

A.7 Implementation and Evaluation Details

Subgraph reuse. The construction procedure in the main text defines local graphs independently. In our implementation, only the finest-layer graphs are built by insertion. For each parent cube, we inherit child local edges and eligible child cross edges, add child entry points, deduplicate and prune candidates, and perform one synchronous refinement round. Reverse edges are inserted and pruned before constructing cross-cube edges. Each parent refinement reads the previous adjacency snapshot, so parallel updates do not change its input. The resulting index retains the same L-layer organization and bounded degrees. With fixed degree and beam parameters and average $O(\log N)$ graph searches, candidate processing adds a dimension-dependent 2^m term, giving $O(NLm(\log N + 2^m))$ construction work. For fixed m, this has the same asymptotic bound as the insertion-based construction in the main text. The construction measurements throughout this paper use this implementation.

Recall–throughput curves. We report native PGFPlots curves generated directly from measured CSVs. Each curve uses 1,000 shared fixed-cardinality queries, $k = 20$, a swept grid of search widths, and the median of three repeats. Lines connect measured settings; they do not interpolate a claimed operating point. The recall axis is cropped at 0.8 and QPS is logarithmic. Every number quoted in this section is the QPS at a 99% recall floor satisfied in every repeat, so the comparison never relies on an extrapolated or averaged recall.

Comparison set and execution host. The comparison set is CubeGraph, original-author KHI , original iRangeGraph, ACORN , and PostFiltering . In the box comparison, all methods are measured on the same evaluation machine, with single-threaded queries, against the same fixed-cardinality workloads and the same ground truth. CubeGraph uses predetermined search for these box curves. No curve combines timings from different machines and no point is transferred from another protocol.

The set therefore covers all three families the paper argues about: global graph search with post-filtering (PostFiltering), attribute-partitioned local graphs (KHI and iRangeGraph), and attribute-aware global graph search (ACORN).

For PostFiltering the swept parameter is the candidate budget handed to the vector search before filtering, because a post-filter that is only allowed to look at k candidates is a straw man; the panels plot recall against throughput for budgets of 20–2048, the same grid every other method uses for its search width. ACORN 's three-repeat curves are shown for the configurations with completed audits; the unavailable configurations are omitted from the panels and from ratios.

Audit scope. KHI uses the author `khi-paper-release` commit `de411a1`; and iRangeGraph uses `3cfa10b`. The 59 pinned author source files pass SHA-256 verification, and the external format adapters preserve the original predicates rather than reimplementing them. All ten KHI configurations and all four native 2D iRangeGraph configurations completed per-query replay audits: returned IDs, predicate membership and recall are recomputed from the retained raw outputs, 192,000 query outputs per configuration. For these box workloads, ACORN , PostFiltering retain aggregate outputs for the original box runs; every plotted configuration passes independent completeness and range re-checks. The native iRangeGraph comparison is restricted to two filtering attributes because that is the only one the author implementation supports; KHI and ACORN supply the 3D and 4D comparison.

ACORN *settings.* ACORN 's curves use $\gamma = 12$, the value the original paper recommends. Coverage is SIFT 2D/3D/4D, YFCC 2D/3D, MSMARCO10M 2D/3D/4D, and WIKI 2D/3D. The additional MSMARCO10M 3D/4D runs reuse the vector index and include per-query returned-ID audits (Appendix A.9).

A.8 Index Construction and Storage

Table 7 covers all ten dataset–dimension configurations. All 44 displayed measured entries have nonzero construction times. Each CubeGraph entry uses its native bottom-up construction with one

Table 7: Native construction time (s) and serialized size (decimal GB), with method/CG ratios in parentheses. CG is 1.00× in each row; a larger ratio means more time or space. A dash denotes an unmeasured configuration. CG uses one native build per configuration; timing scopes and requested parallelism differ (Section A.8).

Dataset	Dim.	Metric	CubeGraph	KHI	iRangeGraph	ACORN	POST
SIFT	2	Time (s)	65.2 (1.00×)	63.3 (0.97×)	433.0 (6.64×)	88.4 (1.36×)	521.1 (8.00×)
		Size (GB)	1.088 (1.00×)	1.399 (1.29×)	1.309 (1.20×)	1.022 (0.94×)	0.140 (0.13×)
SIFT	3	Time (s)	52.0 (1.00×)	63.6 (1.22×)	—	88.4 (1.70×)	534.7 (10.28×)
		Size (GB)	0.812 (1.00×)	1.399 (1.72×)	—	1.022 (1.26×)	0.140 (0.17×)
SIFT	4	Time (s)	41.7 (1.00×)	63.1 (1.51×)	—	88.4 (2.12×)	524.8 (12.58×)
		Size (GB)	0.676 (1.00×)	1.399 (2.07×)	—	1.022 (1.51×)	0.140 (0.21×)
YFCC	2	Time (s)	167.6 (1.00×)	98.6 (0.59×)	836.2 (4.99×)	176.5 (1.05×)	953.9 (5.69×)
		Size (GB)	1.088 (1.00×)	1.338 (1.23×)	1.282 (1.18×)	2.557 (2.35×)	0.140 (0.13×)
YFCC	3	Time (s)	101.2 (1.00×)	97.4 (0.96×)	—	176.5 (1.74×)	937.0 (9.26×)
		Size (GB)	0.812 (1.00×)	1.455 (1.79×)	—	2.557 (3.15×)	0.140 (0.17×)
WIKI	2	Time (s)	159.3 (1.00×)	103.5 (0.65×)	809.3 (5.08×)	215.5 (1.35×)	752.7 (4.72×)
		Size (GB)	1.069 (1.00×)	1.657 (1.55×)	1.574 (1.47×)	2.009 (1.88×)	0.137 (0.13×)
WIKI	3	Time (s)	98.6 (1.00×)	133.0 (1.35×)	—	215.5 (2.19×)	724.1 (7.35×)
		Size (GB)	0.798 (1.00×)	1.655 (2.07×)	—	2.009 (2.52×)	0.137 (0.17×)
MSMARCO10M	2	Time (s)	4,328.6 (1.00×)	3,575.8 (0.83×)	19,366.6 (4.47×)	4,258.6 (0.98×)	10,609.4 (2.45×)
		Size (GB)	10.880 (1.00×)	19.468 (1.79×)	18.420 (1.69×)	46.056 (4.23×)	1.400 (0.13×)
MSMARCO10M	3	Time (s)	3,328.6 (1.00×)	3,574.3 (1.07×)	—	4,258.6 (1.28×)	10,500.2 (3.15×)
		Size (GB)	10.122 (1.00×)	19.468 (1.92×)	—	46.056 (4.55×)	1.400 (0.14×)
MSMARCO10M	4	Time (s)	3,246.3 (1.00×)	3,503.5 (1.08×)	—	4,258.6 (1.31×)	10,644.0 (3.28×)
		Size (GB)	8.966 (1.00×)	19.467 (2.17×)	—	46.056 (5.14×)	1.400 (0.16×)

refinement round, 128 requested threads, $M = 16$, $ef_c = 200$ and refinement width 64. The controlled physical ablation reports independent three-build medians separately. ACORN reuses one vector index across metadata dimensions of a dataset, so its repeated entries refer to the same artifact and construction time. Unmeasured configurations are marked "—".

Construction scope. These are observations from native build scripts and same-configuration reruns on the evaluation machine, not an equal-thread construction benchmark. CubeGraph's timer wraps `build_index`, including its input reads and excluding serialization; KHI and iRangeGraph report their internal construction timers, excluding saving; ACORN times vector insertion, excluding loading and serialization. POST times insertion plus saving after input loading. The build settings also differ: CubeGraph requests 128 OpenMP threads, KHI and iRangeGraph request 32 builder threads, ACORN requests 72, and POST inserts points serially. These requested counts are not measurements of effective parallelism. We therefore report native observations and method/CG ratios; these ratios do not isolate equal-thread algorithmic speedups. CubeGraph's three-repeat ablations are identified separately from single builds.

Storage scope. Sizes are inventoried logical file lengths (fresh outputs for rerun entries) in decimal GB, not allocated disk blocks, peak RSS, or total serving memory. CubeGraph includes the directory file and every layer file; KHI includes both tree and graph files. For the other methods we count the native serialized index file. Source vectors, external attribute files, converted-input copies, query masks, and ground truth are not added to these file sizes. Serialization formats differ: an implementation may embed data that another reads externally, so these values do not establish a uniform total-memory ranking. The three MSMARCO10M 2D indexes in main-paper Table 5 all exclude source vectors; CubeGraph includes per-point attributes and its layer directory, while KHI includes its tree. For example, the POST graph reads its vectors from an external array. ACORN's vector index is reused across attribute dimensions of the same dataset; its repeated sizes and available build times denote that shared artifact, not independent builds. Peak RSS for the controlled physical ablations is reported separately in Table 1.

A.9 Layer Sensitivity and ACORN Coverage

ACORN coverage. The additional MSMARCO10M 3D/4D runs reuse the same $\gamma = 12$, $M = 32$, $M_\beta = 64$ vector index as 2D. Each configuration uses the frozen shared box predicates, 1,000 queries, $k = 20$, nine search widths from 20 to 4096, and three repeats. Query timing excludes mask preparation and validation. Returned IDs were independently checked for eligibility, uniqueness, and recall against the previously audited ground truth. Main-paper Figure 4 includes every curve; Table 8 gives the operating points.

Table 8: ACORN high-dimensional operating points. A number is QPS at the 99% floor in all three repeats; N/R is followed by the highest measured recall, in percent.

Dimension	1%	2%	5%	10%
3D	N/R (93.78)	13.3	59.0	60.0
4D	N/R (94.67)	N/R (98.96)	59.1	61.7

Controlled layer retention. We retain the finest $L = 1, \ldots, 6$ native layers on SIFT1M and MSMARCO10M 2D. The finest resolution, vectors, attributes, graph edges, and cross edges are identical across variants; increasing L adds coarser layers. The native layer selector operates on the retained layers without modification. This isolates retention of coarse layers from changes in finest cube size. Each variant uses four qualifying fractions, eight search widths from 20 to 2048, and three repeats on 1,000 shared queries ($k = 20$). We serialize a subset directory and reference the unchanged layer files; Table 9 counts their logical sizes, not incremental disk allocation. No new graph construction is timed.

Table 9: Serialized index size in decimal GB for the retained-layer variants, excluding source vectors.

L	1	2	3	4	5	6
SIFT1M	0.188	0.368	0.548	0.728	0.908	1.088
MSMARCO10M	1.880	3.680	5.480	7.280	9.080	10.880

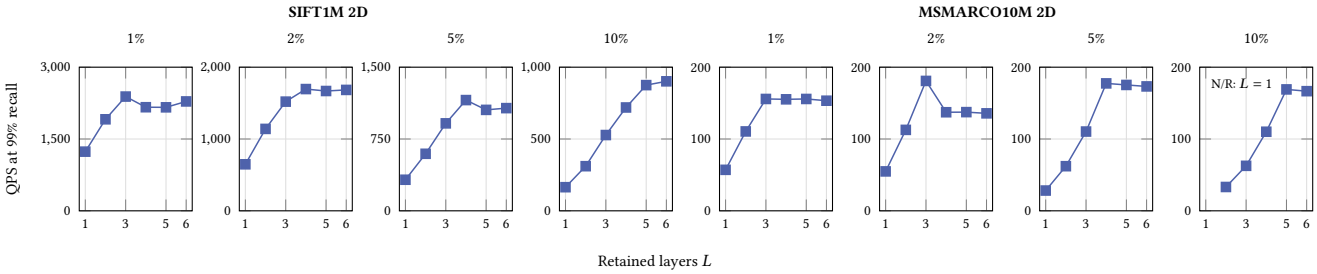


Figure 5: Layer-retention sensitivity with fixed finest resolution (linear axes). Each index keeps native layers $6 - L, \ldots, 5$ of the original six-layer index, with unchanged graphs and cross edges. At each qualifying fraction, QPS is the median at the smallest measured width reaching 99% recall in all three repeats. N/R means no measured width reaches that floor.

Results and limits. For each dataset, $L = 5$ and $L = 6$ return byte-identical IDs in all 96 measured cells. Five layers save 16.5% serialized space. Their measured operating-point QPS differs from six layers by up to 5.4%. This statement applies to the frozen workloads, not arbitrary predicates. On MSMARCO10M, five layers occupy 9.080 GB, 53.4%/50.7% below the native KHI/iRangeGraph files; the main comparison uses six layers. Figure 5 reports every measured space–throughput trade-off, including reversals and widths that fail the recall floor. No single retained-layer count is assumed optimal for all predicates.

A.10 Applicability of the Analysis and Evaluation

Geometry, density, and traversal. The cube-count proposition holds for any object distribution when the preferred resolution exists. It controls the number of intersected grid cells, even if their occupancies are highly unequal. The elastic-factor statement instead concerns the objects in a full geometric cover.

For a joint density bounded by $0 < f_{\min} \le f(\mathbf{s}) \le f_{\max}$ on U, direct integration gives

$$\frac{\mu(\phi)}{\mu(U)} \ge \frac{f_{\min}}{f_{\max}} \frac{\mathrm{Vol}(\phi)}{\mathrm{Vol}(U)}.$$

Without such a density condition, the volume ratio alone gives no positive distribution-independent lower bound on the eligible fraction. A large candidate-pool fraction also need not imply a large conditional success probability during vector-guided graph traversal. This is why the query-work theorem retains c as an assumption rather than substituting the ball-volume bound.

Shape and resolution. The query interface assumes that a bounding box and a representative point are supplied or computable for the numeric region, in addition to point-membership evaluation. A Boolean evaluation oracle alone need not provide this geometric information. The longest side of $B(\phi)$ is an inexpensive scale descriptor. A box and the same box with an interior exclusion have the same scale, but their qualifying sets and fill ratios differ. A narrow diagonal polygon may have a large bounding box and low fill ratio even with few vertices. In these cases the selected layer can still admit many invalid candidates. More vertices may make cube–polygon intersection expensive without changing selectivity. On-the-fly discovery avoids enumerating all intersections in advance, but a finite beam may miss components separated by empty cubes. Exact predicate checks prevent invalid outputs; they do not imply complete recovery of eligible neighbors.

Table 10: Search modes at 99% recall in every repeat. QPS is the median of three runs. Ratio = predetermined/on-the-fly; values above one favor predetermined.

Dataset	Fraction	Pred. QPS	Fly QPS	Ratio
SIFT1M	1%	2,391.4	2,284.7	1.047×
SIFT1M	2%	1,812.1	1,683.6	1.076×
SIFT1M	5%	1,173.7	1,072.5	1.094×
SIFT1M	10%	963.5	901.6	1.069×
MSMARCO10M	1%	152.9	153.3	0.997×
MSMARCO10M	2%	135.4	135.7	0.998×
MSMARCO10M	5%	174.1	173.2	1.005×
MSMARCO10M	10%	166.8	166.6	1.001×

Coverage of the empirical evidence. The shared box comparisons include real attributes and synthetic uniform attributes, while the five-distribution study in A.3 varies synthetic distributions for CubeGraph. It is not a controlled skew sweep against KHI or an R-tree baseline. The ball and exclusion-region experiments compare methods at fixed qualifying counts, but do not independently sweep polygon vertex count, bounding-box fill ratio, or exclusion complexity. The ACORN comparison uses $\gamma = 12$ throughout and does not establish performance under per-workload retuning. These are limits of the current evidence, not consequences of the geometric proposition.

The insert-only study measures batches into a 100K-object initial index and reports post-update recall and throughput. It does not establish large-scale deletion performance or sustained mixed-update behavior. Similarly, the 6D/8D probes concern construction costs; the query comparisons stop at 4D. The depth study reports both storage and query effects for its frozen workloads. Further comparisons under controlled skew, shape-fit variation, and larger update streams are needed to characterize these regimes.

A.11 Predetermined versus On-the-Fly Search

Controlled comparison. We compare the two discovery modes on SIFT1M and MSMARCO10M 2D using the same six-layer graph files, base/query vectors, box predicates, exact ground truth, and widths. Both modes execute single-threaded, with 20 warm-up queries before each of three repeats of 1,000 queries. The predetermined pass uses the original full index. The on-the-fly pass uses the six-layer retention artifact: its directory is byte-identical to the original, and its six graph files are links to the same files. Independent audits

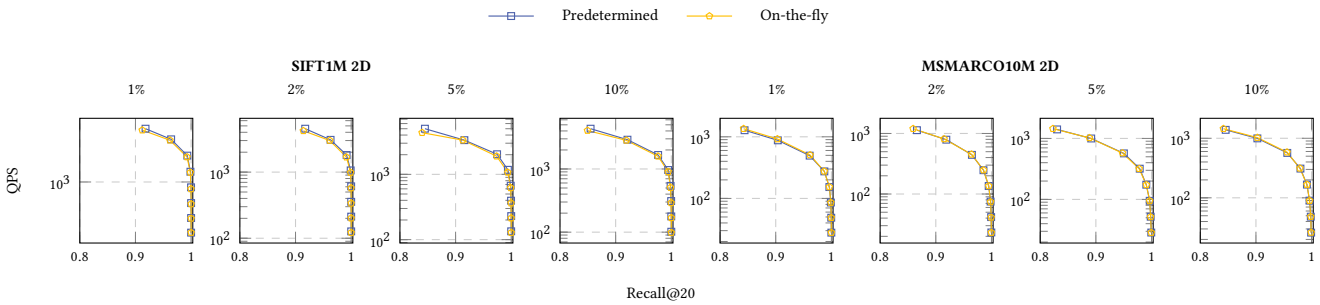


Figure 6: Predetermined versus on-the-fly search on identical six-layer indexes and identical box workloads. Three-repeat median curves; all widths are retained. These are separate sequential timing passes.

check all 384 returned-ID files for eligibility, uniqueness, and recall; workload hashes agree between modes.

Results. Figure 6 retains every width, while Table 10 selects the smallest width reaching 99% recall in all repeats for each mode independently. The selected widths coincide: 64/64/128/128 on SIFT1M and 256 at all four fractions on MSMARCO10M. Predetermined search yields 4.7–9.4% higher throughput on SIFT1M. On MSMARCO10M the ratios are 0.997–1.005, indicating very similar measured throughput. These runs were sequential passes, rather than interleaved paired trials; small timing differences should not be interpreted as a statistically established advantage.

Scope. This comparison isolates the search-mode choice on the evaluated box workloads. It does not measure the cost of an expensive cube–polygon intersection test or establish an advantage for either mode on disconnected predicates. The complex-predicate evaluation in A.4 uses on-the-fly search throughout and must not be interpreted as a cross-mode comparison.